

ARM PrimeCell

External Bus Interface (PL220)

Design Manual

ARM PrimeCell External Bus Interface (EBI) Design Manual

© Copyright ARM Limited 2002. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.
All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com/>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.1.1	Current status and anticipated changes	1
1.1.2	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
1.4	Conventions used in this document	2
1.4.1	Signals	2
1.4.2	Bytes, Half-words and Words	2
1.4.3	Bits, bytes, K and M	2
2	SCOPE	2
3	INTRODUCTION	2
4	EBI DESIGN OVERVIEW	3
4.1	Signal Description	3
4.2	Design Hierarchy	5
4.3	EBI Block Partitioning	5
4.4	EBI Arbitration and Control block	7
4.5	EBI Multiplexer Block	8
5	EBI IMPLEMENTATION DETAILS	10
5.1	EBI Top Level Module (Ebi)	10
5.2	EBI Arbitration and Control block (EbiArbCntl)	10
5.3	EBI Multiplexer Block (EbiMux)	13
6	EBI VERIFICATION TESTBENCH	14
6.1	EBI VHDL Verification Testbench	14
6.1.1	Testbenches	16
6.1.2	Unit Under Test	16
6.1.3	Bus Master Protocol Checker	17
6.1.4	AHB Master/Arbiter Model	17
6.1.5	Decoder	17
6.1.6	Default Slave	17
6.1.7	Protocol Checker	17

6.1.8	Trickbox	17
6.1.9	Simulation environment	18
6.2	EBI Verilog Verification Testbench	19
6.2.1	Testbenches	20
6.2.2	Unit Under Test	20
6.2.3	Bus Master Protocol Checker	21
6.2.4	AHB Master/Arbiter Model	21
6.2.5	Decoder	21
6.2.6	Default Slave	21
6.2.7	Protocol Checker	21
6.2.8	Trickbox	21
6.2.9	Simulation Environment	22
6.3	Trickbox Description	23
6.3.1	Trickbox Signal Description	24
6.3.2	AHB Interface Block	27
6.3.3	Register Block	28
6.3.4	Clock and Reset Generator	31
6.3.5	EBI Mirror Trick Box	31
6.3.6	Comparator	35
6.3.7	Protocol checker	36
6.3.8	EBI Request Indicator	36
6.3.9	Trickbox register summary	37
7	VERIFICATION TESTS	42
7.1	FUNCTIONAL TESTS	42
7.1.1	EBIMCkEqHCLK Test	42
7.1.2	EBIMCk1Eq2HCLK Test	42
7.1.3	EBIMCk2Eq2HCLK Test	43
7.1.4	EBIMCk3Eq2HCLK Test	43
7.1.5	EBIMCk1Mck2Eq2HCLK Test	44
7.1.6	EBIMCk1Mck3Eq2HCLK Test	44
7.1.7	EBIMCk2Mck3Eq2HCLK Test	45
7.1.8	EBIM1M2M3Eq2HCLK Test	45
7.1.9	EBIRandom Test	46

1 ABOUT THIS DOCUMENT

1.1 Change history

1.1.1 Current status and anticipated changes

This document has been reviewed and the review comments have been incorporated.

1.1.2 Change history

Issue	Date	Change
A01	15 th May, 2002	First Draft
A02	1 Dec. 2002	Review comments incorporated

1.2 References

This document refers to the following documents.

Ref.	Doc. No.	Title
1	ARM IHI0011	AMBA Specification Rev2.0
2	ARM DDI 0249A	ARM PrimeCell External Bus Interface (PL220) Technical Reference Manual
3	ARM DUI 0006	AMBA Compliance Test Suite User Guide
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High performance Bus, an AMBA bus designed for high-performance peripherals.
AHB master	A device that can send out requests over an AHB bus.
AHB slave	A device that fulfils requests provided by an AHB master.
AMBA	Advanced Microcontroller Bus Architecture.
EBI	External Bus Interface
Mux	Multiplexer

1.4 Conventions used in this document

1.4.1 Signals

- Signal names are shown in bold font type.
- Active low signals are prefixed by a lower case 'n'. In the case of AHB signals by 'Hn', or post fixed by 'n'.
- AHB signals are prefixed by an upper case 'H'.
- When a signal is described as being 'Asserted', the actual level depends on whether the signal is active HIGH or active LOW. 'Asserted' means HIGH for active high signals and LOW for active low signals.

1.4.2 Bytes, Half-words and Words

- A byte is 8 bits.
- A half-word is two bytes i.e. 16 bits.
- A word is 4 bytes i.e. 32 bits.

1.4.3 Bits, bytes, K and M

- The suffix 'B' (upper case) is used to indicate BYTES.
- The suffix 'b' (lower case) is used to indicate BITS.
- When used to indicate an amount of memory, the suffix 'k' means 1024.
- When used to indicate a frequency, the suffix 'k' means 1000.
- When used to indicate an amount of memory, the suffix 'M' means $1024^2 = 1048576$.
- When used to indicate a frequency, the suffix 'M' means 1,000,000.

2 SCOPE

This document describes the design and implementation of the ARM PrimeCell EBI (External Bus Interface). The design description is split between two sections – Section 4 and Section 5. The test benches and the test programs for functional verification are described in Section 6 and section 7

Section 4 presents an overview of the EBI design, describing the functional partitioning of the EBI and the signal interconnections. Section 5 presents detailed descriptions on the EBI design. Section 6 describes the test bench and section 7 describes the test programs used to generate test vectors for functional verification of the EBI.

3 INTRODUCTION

The EBI is a part of a library of reusable IP blocks, which can be easily integrated into a typical ASIC design flow.

For further information on this block refer to the ARM PrimeCell External Bus Interface Technical Reference Manual [Ref.2].

The AMBA Compliance Test Suite User Guide [Ref.3] is a source for further information about AMBA bus compliance test benches.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

4 EBI DESIGN OVERVIEW

The External Bus interface can be connected to three memory controller. It arbitrates the requests coming from three controllers and asserts **EBIGNT** to the highest priority controller. Also it multiplexes the address, data and data enable lines from three separate controllers on to the common address, data and data enable pins of the chip.

4.1 Signal Description

The following tables describe the External Bus Interface signals. **Table 4.1-1** is the system side signals and **Table 4.1-2** lists the Scan test signals.

Name	Type	Source/Destination	Description
EBICK	Input	Clock source	EBI clock
nPOR	Input	Reset Controller	Power on reset (active low).
EBIREQ1	Input	From the Controller 1	EBI request for port 1. Active high
EBIGNT1	Output	To the Controller 1	EBI grant for port 1. Active high
EBIBACKOFF1	Output	To the Controller 1	EBI backoff signal for port 1. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR1[31:0]	Input	From the Controller 1	EBI address for port 1.
EBIDATA1[31:0]	Input	From the Controller 1	EBI data for port 1
nEBIDATAEN1[3:0]	Input	From the Controller 1	Data enable for port 1.
EBITIMEOUTVALUE1[9:0]	Input	System Controller	Gives the value to be loaded into timeout counter for port 1. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ2	Input	From the Controller 2	EBI request for port 2. Active high
EBIGNT2	Output	To the Controller 2	EBI grant for port 2. Active high
EBIBACKOFF2	Output	To the Controller 2	EBI backoff signal for port 2. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR2[31:0]	Input	From the Controller 2	EBI address for port 2.
EBIDATA2[31:0]	Input	From the Controller 2	EBI data for port 2
nEBIDATAEN2[3:0]	Input	From the Controller 2	Data enable for port 2.
EBITIMEOUTVALUE2[9:0]	Input	System Controller	Gives the value to be loaded into

Name	Type	Source/Destination	Description
			timeout counter for port 2. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ3	Input	From the Controller 3	EBI request for port 3. Active high
EBIGNT3	Output	To the Controller 3	EBI grant for port 3. Active high
EBIBACKOFF3	Output	To the Controller 3	EBI backoff signal for port 3. Active high indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR3[31:0]	Input	From the Controller 3	EBI address for port 3.
EBIDATA3[31:0]	Input	From the Controller 3	EBI data for port 3
nEBIDATAEN3[3:0]	Input	From the Controller 3	Data enable for port 3.
EBITIMEOUTVALUE3[9:0]	Input	System Controller	Gives the value to be loaded into timeout counter for port 3. Can be tied off to a fixed value or supplied by the system controller.
EBIDATAIN[31:0]	Output	To all three Controllers	Data connected to all the memory controllers
EBIEXTADDRROUT[31:0]	Output	To Pads	Address output to pads
EBIEXTDATAOUT[31:0]	Output	To Pads	Data output to data pads
nEBIEXTDATAEN[3:0]	Output	To Pads	Data enable to data pads
EBIEXTDATAIN[31:0]	Input	From Pads	Data input from data pads

Table 4.1-1 System side signals

Name	Type	Source/Destination	Description
SCANENABLE	Input	From the Test Controller	Scan enable test signal. Indicates that the circuit is in scan-test mode.
SCANINEBICLK	Input	From the Test Controller	EBICLK domain Scan data input for clocking in the test pattern in scan-test mode.
SCANOUTEBICLK	Output	To the Test Controller	EBICLK domain Scan data output for observing the flip-flop contents in scan test mode.

Table 4.1-2 Scan test signals

4.2 Design Hierarchy

The hierarchy in the EBI is shown in **Figure 4.2-1**. A short description about the functionality of each of the sub-modules is also given within brackets.

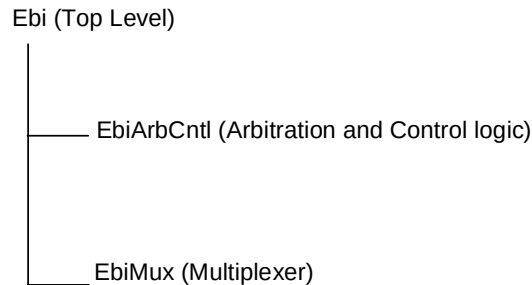


Figure 4.2-1 EBI Design Hierarchy

4.3 EBI Block Partitioning

At the topmost level, the EBI consists of the following blocks:

- EBI Arbitration and Control Block (EbiArbCntl)
- EBI Multiplexer Block (EbiMux)

The Arbitration and Control Block arbitrates the requests coming from three controllers and gives **EBIGNT** to the highest priority. The priority is based on the timeout counter as well as the round robin arbitration algorithm. It also generates the **EBIBACKOFF** signal.

The Multiplexer Block multiplexes the address, data and data enable lines from three separate controllers on to the common address, data and data enable pins of the chip.

A block schematic of the ARM PrimeCell External Bus Interface is shown in **Figure 4.3-1**.

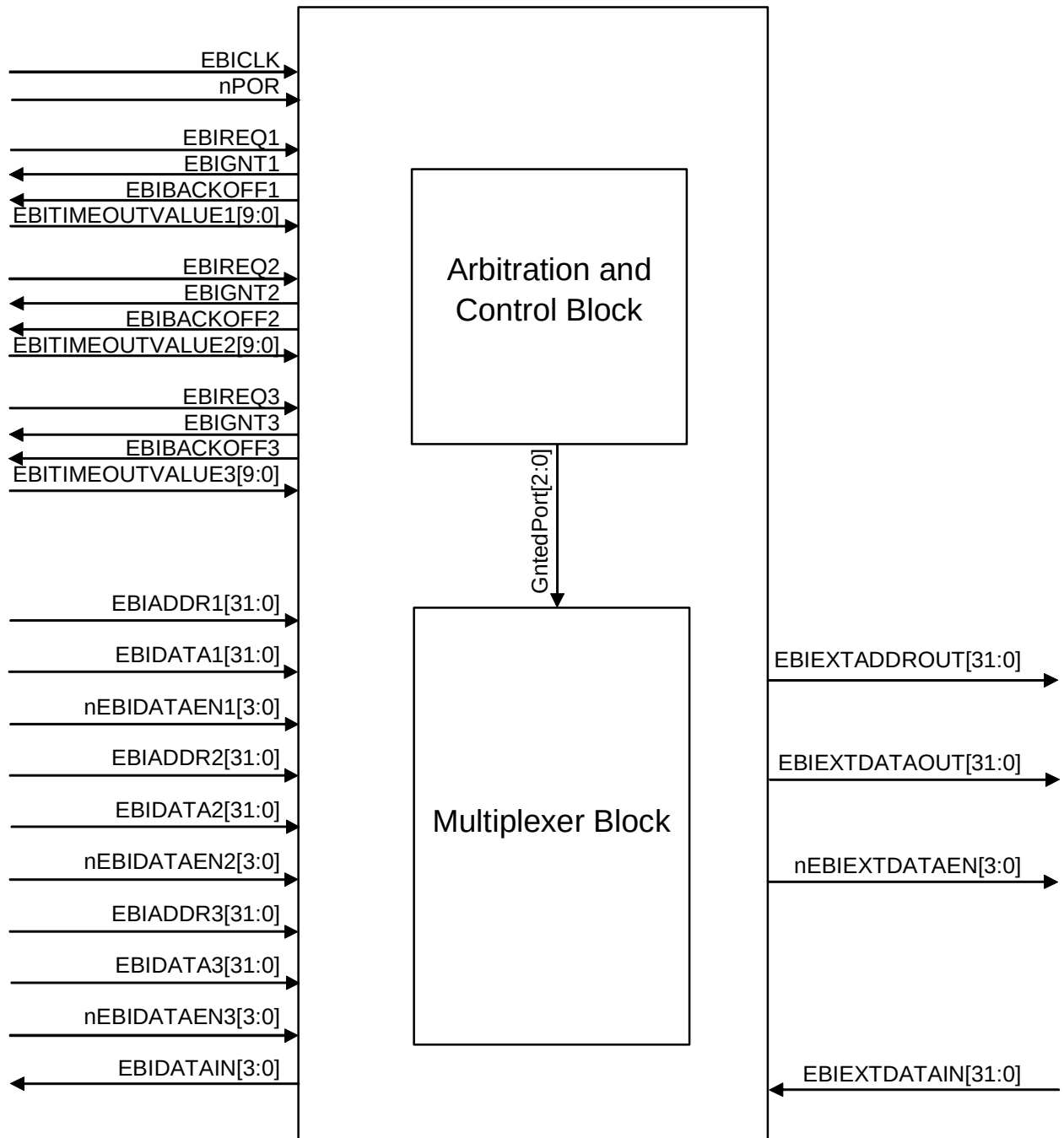


Figure 4.3-1 External Bus Interface Block Diagram

The EBI Arbitration and Control block consists of the following major logic:

- Timeout Counters
- Backoff signal controller
- Round Robin Priority Arbiter
- Grant Control logic

4.4 EBI Arbitration and Control block

The port level connections of the EBI Arbitration and Control block (EbiArbCntl) are shown in **Figure 4.4-1**

Arbitration and Control Block does the arbitration of the requests coming from three controllers and gives **EBIGNTx** to the highest priority controller. Each port has a 10 bit counter which will be loaded with a value from the **EBITIMEOUTVALUE[9:0]** input whenever a request is made for the bus. If the signal **EBITIMEOUTVALUE[9:0]** is zero then the counter will not be started for the port. When the counter reaches 0 then the controller which is currently granted the bus, will be requested to release the bus by asserting the **EBIBACKOFF** signal. The first port counter to reach zero will be given the highest priority for the bus, followed by the second device. If more than one controller issues a request for the bus at the same time then a round robin arbitration scheme will be used to decide which controller should be granted the bus. When no device is requesting the use of the bus then the most recently granted controller will continue to be granted the bus. At reset, controller-1 will be granted the bus.

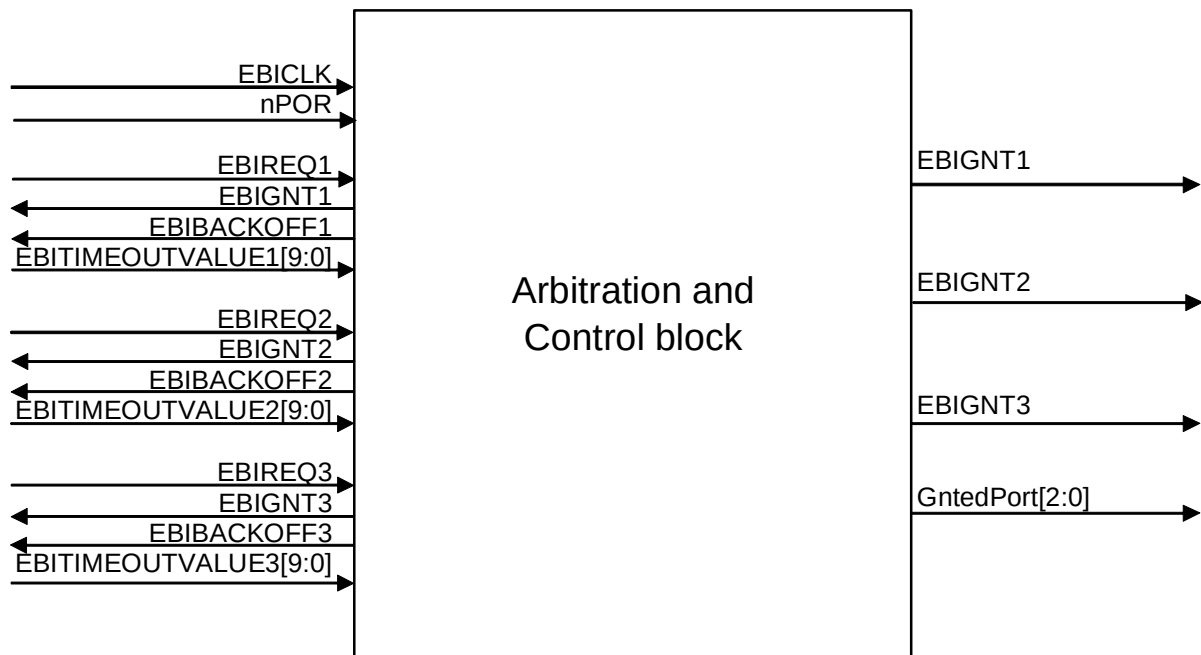


Figure 4.4-1 Block Inputs and Outputs of EbiArbCntl

The list of inputs to the EbiArbCntl module is shown in **Table 4.4-1** and the list of outputs from the EbiArbCntl module is shown in **Table 4.4-2**

Signal Name	Source	Description
EBICLK	Clock source	EBI clock
nPOR	Reset Controller	Power on reset (active low).
EBIREQ1	From the Controller 1	EBI request for port 1. Active high
EBITIMEOUTVALUE1[9:0]	System Controller	Gives the value to be loaded into timeout counter for port 1. Can be tied off to a fixed value or supplied by the system controller.

Signal Name	Source	Description
EBIREQ2	From the Controller 2	EBI request for port 2. Active high
EBITIMEOUTVALUE2[9:0]	System Controller	Gives the value to be loaded into timeout counter for port 2. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ3	From the Controller 3	EBI request for port 3. Active high
EBITIMEOUTVALUE3[9:0]	System Controller	Gives the value to be loaded into timeout counter for port 3. Can be tied off to a fixed value or supplied by the system controller.

Table 4.4-1 Block Inputs of EbiArbCntl

Name	Destination	Description
EBIGNT1	To the Controller 1	EBI grant for port 1. Active high
EBIBACKOFF1	To the Controller 1	EBI backoff signal for port 1. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIGNT2	To the Controller 2	EBI grant for port 2. Active high
EBIBACKOFF2	To the Controller 2	EBI backoff signal for port 2. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIGNT3	To the Controller 3	EBI grant for port the 3. Active high
EBIBACKOFF3	To the Controller 3	EBI backoff signal for port 3. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
GntedPort[2:0]	To the EBI Multiplexer Block (EbiMux)	Granted port. Indicates to the multiplexer which port being currently granted. Each bit of this corresponds to an EBIGNTx port

Table 4.4-2 Block Outputs of EbiArbCntl

4.5 EBI Multiplexer Block

The multiplexer provides highly constrained combinational paths for address, data and data enable signals. It contains a set of flip-flops to store the default values when no controller is selecting the bus

Figure 4.5-1 shows the port level block diagram of EBI Multiplexer Block (EbiMux).

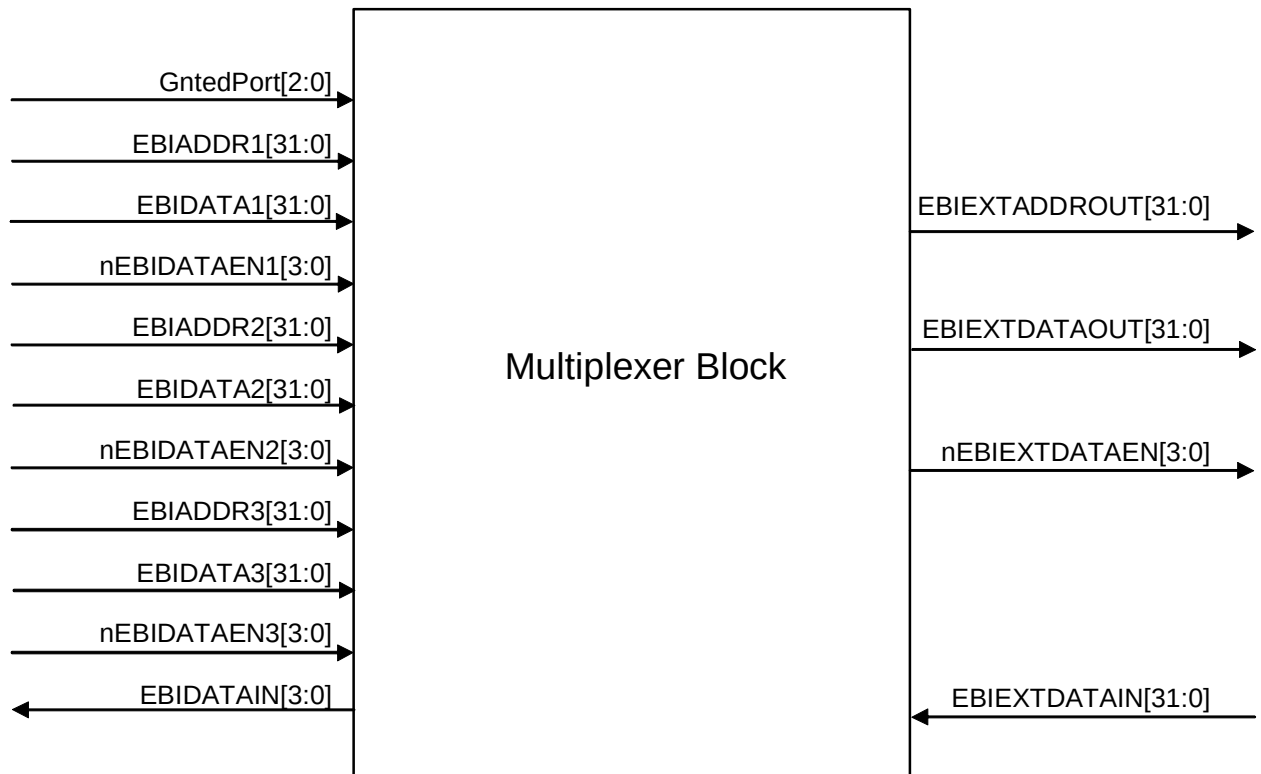


Figure 4.5-1 Block Inputs and Outputs of EbiMux

The list of inputs and outputs of the EbiMux module are shown in **Table 4.5-1** and **Table 4.5-2**

Name	Source	Description
GntedPort[2:0]	From the EBI Arbitration and Control block (EbiArbCntl)	Granted port. Indicates to the multiplexer which port being currently granted. Each bit of this corresponds to an EBIGNTx port
EBIADDR1[31:0]	From the Controller 1	EBI address for port 1.
EBIDATA1[31:0]	From the Controller 1	EBI data for port 1
nEBIDATAEN1[3:0]	From the Controller 1	Data enable for port 1.
EBIADDR2[31:0]	From the Controller 2	EBI address for port 2.
EBIDATA2[31:0]	From the Controller 2	EBI data for port 2
nEBIDATAEN2[3:0]	From the Controller 2	Data enable for port 2.
EBIADDR3[31:0]	From the Controller 3	EBI address for port 3.
EBIDATA3[31:0]	From the Controller 3	EBI data for port 3

Name	Source	Description
nEBIDATAEN3[3:0]	From the Controller 3	Data enable for port 3.
EBIEXTDATAIN[31:0]	From Pads	Data input from data pads

Table 4.5-1 Block Inputs of EbiMux

Name	Destination	Description
EBIEXTADDRROUT[31:0]	To Pads	Address output to pads
EBIEXTDATAOUT[31:0]	To Pads	Data output to data pads
nEBIEXTDATAEN[3:0]	To Pads	Data enable to data pads
EBIDATAIN[31:0]	To all three Controllers	Data connected to all the memory controllers

Table 4.5-2 Block Outputs of EbiMux

5 EBI IMPLEMENTATION DETAILS

In this section, the implementation details of the blocks listed in section 4 are provided.

5.1 EBI Top Level Module (Ebi)

The top-level module is a structural block which instantiates and interconnects the two main functional blocks in the EBI – the EBI Arbitration and Control block and the EBI Multiplexer block. All the functional blocks in the EBI are described in the following sections.

5.2 EBI Arbitration and Control block (EbiArbCntl)

A block diagram of Arbitration and Control Block is shown in **Figure 5.2-1**. Intention of this figure is to explain the flow inside the Arbitration and Control Block. Figure contains relevant control signals.

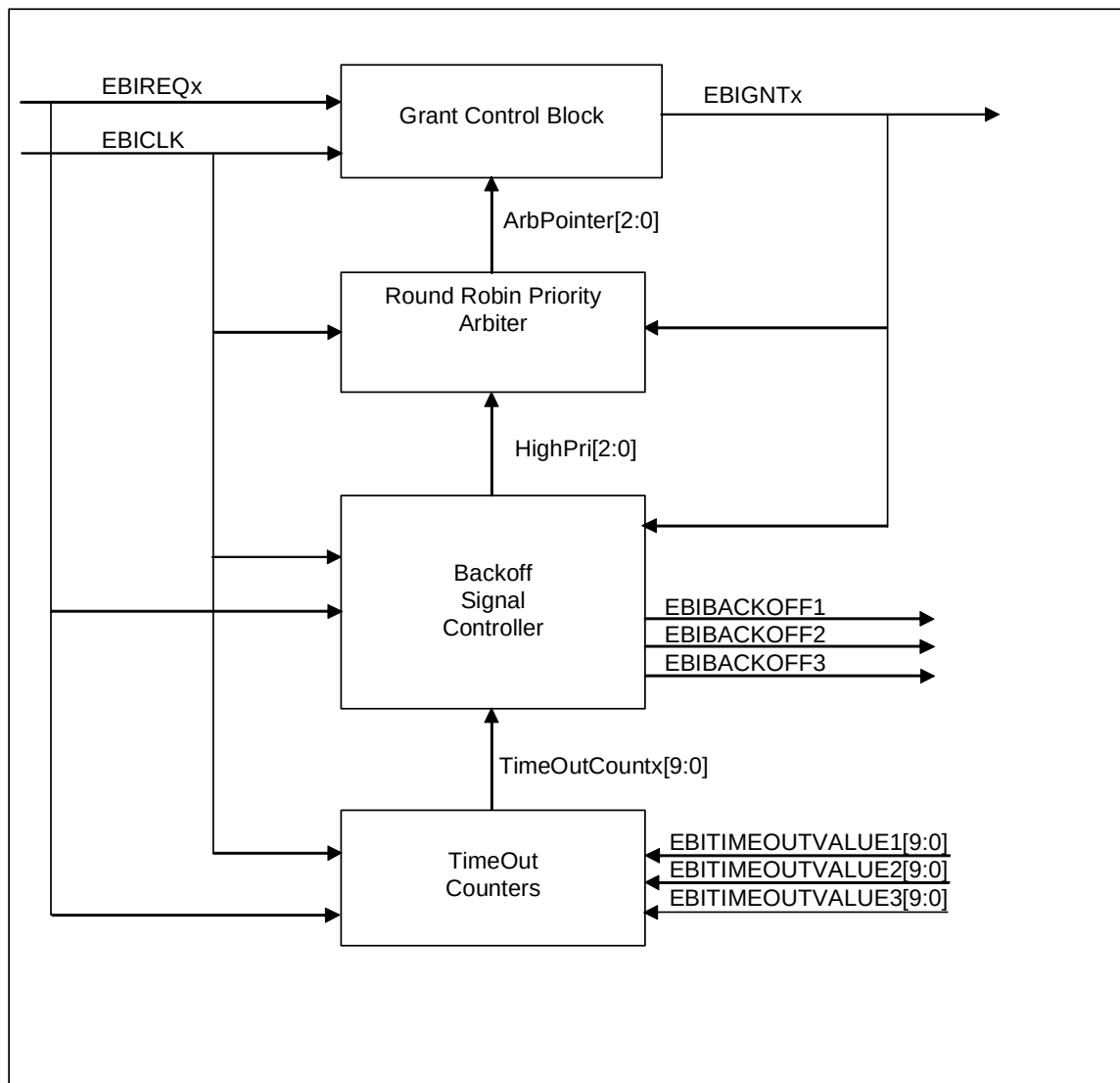


Figure 5.2-1 Arbitration and Control Block

Figure 5.2-1 shows the main components of the EBI Arbitration and Control block, which are:

- Timeout Counters
- Backoff signal controller
- Round Robin Priority Arbiter
- Grant Control logic

Timeout counters

The logic contains three counters. These counters are loaded with the value on **EBITIMEOUTVALUEx[9:0]** from the system controller on assertion of **EBIREQx**. It starts to countdown at every rising edge of **EBICLK**. If the signal **EBITIMEOUTVALUEx[9:0]** is zero then the counter will not be started for the port. **TimeOutCountx[9:0]** is driven to the Backoff controller. These counters determine the assertion of **EBIBACKOFF** and **HighPri** in the Backoff controller logic.

Backoff signal controller

This logic is responsible for generating the **EBIBACKOFF** and **HighPri** signals. It uses the **EBIGNTx** signal from the Grant control logic, and the **TimeOutCountx[9:0]** from the Timeout counters to generate these signals.

EbiReqxGnt and **EbiReqActx** signals are generated to detect an active grant and request. The **EbiReqxGnt** signal is asserted only if the **EBIGNTx** and its corresponding **EBIREQx** is asserted. It avoids generation of **EBIBACKOFF** to a port that has been given the grant by default.

The **EbiReqActx** signal is asserted in any one of the following conditions

- If **EBIREQx** is asserted and its corresponding **TimeOutCountx[9:0]** value is one and the signal **EBIREQxQ** is one. (the signal **EBIREQxQ** is the registered version of the **EBIREQx**).
- If **EBIREQx** is asserted and its corresponding **EBITIMEOUTVALUEx[9:0]** is zero.

The **HighPri** signal indicates that controller whose **TimeOutCountx[9:0]** counted down to zero first when another controller had been granted. When the value on **TimeOutCountx[9:0]** reaches zero then the port that is currently granted the bus will be requested to release the bus by asserting its **EBIBACKOFF** signal. The **EBIBACKOFF** signal is generated by using the signal **EbiReqActx** and the **EbiReqxGnt** signals. The **EBIBACKOFF** for the granted port will be asserted when **EbiReqActx** signal of any other port has been asserted. The port whose counter reaches zero first will be given the highest priority for the bus, followed by the second port. If two counters become zero at the same time, then **HighPri** is cleared and the round robin algorithm will determine the highest priority. The **HighPri** signal is driven to the Round robin priority arbiter logic where it overwrites the priority determined by the round robin algorithm.

Round Robin Priority Arbiter

The Priority of the controller is determined by the round robin algorithm only in the following conditions

- When two controllers simultaneously request the External Bus Interface while another controller is granted by default.
- When two Timeout counters countdown to zero at the same time when another controller has been granted.

In all other conditions, priority is determined by the **HighPri** signal. Round Robin Priority Arbiter changes the **NxtArbPointer**, which is the D-input of the **ArbPointer** to a value determined by **EBIGNTx**, **ArbPointer** and the **HighPri** signal. It uses the least recently used (LRU) algorithm where the least recently used controller gets the highest priority and most recently used controller gets the least priority. For example, if the order of priority is Controller 1, Controller 2 and Controller 3 and Controller 2 is granted, then the priority order becomes Controller 1, Controller 3 and Controller 2, making Controller 2 the least priority and the order between the other two ports is maintained. The **ArbPointer** is re-assigned to the value in such a way that it should point to the new priority sequence in which the highest priority should be given to the controller indicated by **HighPri** and it should maintain the priority of the other two controllers. Since **HighPri** is cleared if two counters expire simultaneously, the priority sequence determined by the round robin algorithm will take effect in such a case.

Grant Control Logic

Grant Control logic contains the logic to generate the grants to the three ports. This logic gets input **NxtArbPointer** from the Round Robin Priority Arbiter logic. If only one controller requests the bus, then the Grant control logic gives the grant to that controller. If more than one controller raises the request at the same time, then priority of the controllers is determined by the signal **NxtArbPointer**. The **NxtArbPointer** selects the particular priority sequence among six possible priority sequences. The values of **ArbPointer** for the six possible priority sequences are listed below

- 000: Controller 1 has the highest priority followed by Controller 2 and then Controller 3
- 001: Controller 2 has the highest priority followed by Controller 3 and then Controller 1
- 010: Controller 3 has the highest priority followed by Controller 1 and then Controller 2
- 011: Controller 1 has the highest priority followed by Controller 3 and then Controller 2
- 100: Controller 2 has the highest priority followed by Controller 1 and then Controller 3

- 101: Controller 3 has the highest priority followed by Controller 2 and then Controller 1

The granted port is de-granted only if that port de-asserts its request and any other port has requested the grant. If no other port requests, the Grant control logic keeps the grant asserted to the same port even though the controller de-asserts its request. This helps to save one clock if the same controller requests the bus again. The granted port gets highest priority when it again asserts its request with some other controller.

5.3 EBI Multiplexer Block (EbiMux)

The multiplexer provides highly constrained combinational paths for address, data and data enable signals. It contains a set of flip-flops to store the default values when no controller is selecting the bus

The block diagram of Multiplexer shown in **Figure 5.3-1**

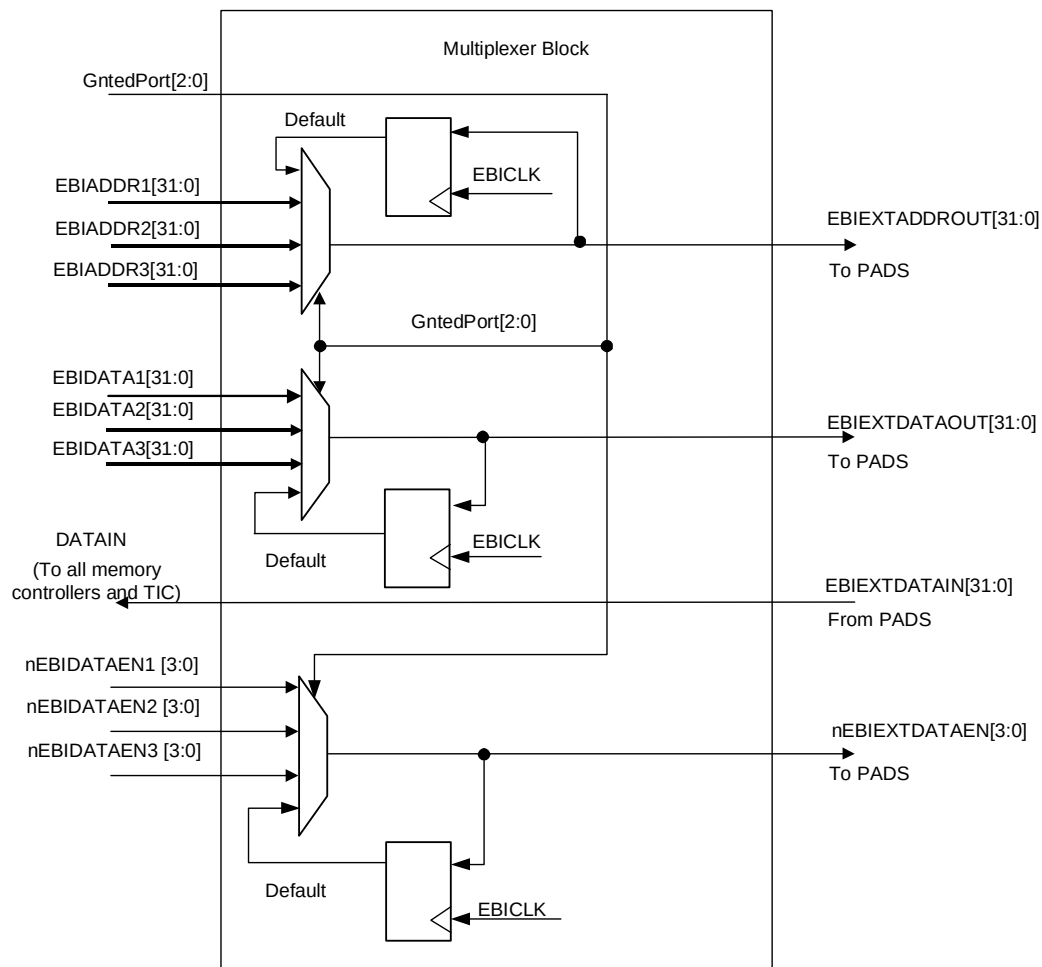


Figure 5.3-1 Multiplexer block diagram

The multiplexers are used for multiplexing the Address, dataout and data enable signals corresponding to Bus grant signals. Three separate multiplexers are used to multiplex the Address, dataout and data enable signals. A set of flip-flops stores the default values when no device controller is selecting the bus.

The data, address and data enable are not registered inside the External Bus Interface but are instead multiplexed to the pads as described above. This means that there is an extra delay in the output path for these signals. There is no

multiplexer in the input data path (*EXTEBIDATAIN*) from the memory and this is connected directly to all the controllers.

6 EBI VERIFICATION TESTBENCH

The EBI Functional Verification testbench is an AHB BusTalk testbench. The VHDL and Verilog BusTalk testbench files reside in the *verification/vhdl* and *verification/verilog* directories respectively.

6.1 EBI VHDL Verification Testbench

Figure 6.1-1 shows the test environment of EBI. The Trickbox is connected to AHB bus. The slave Testbench is responsible for programming the slave registers in Trickbox. The Testbench is interfaced to the Trickbox via AHB Bus. The UUT is interfaced with Trickbox through non-AMBA signals. The Trickbox also does all the protocol checking and drives the non-AMBA signals to the EBI.

Figure 6.1-2 illustrates the Bustalk VHDL testbench for running test vectors.

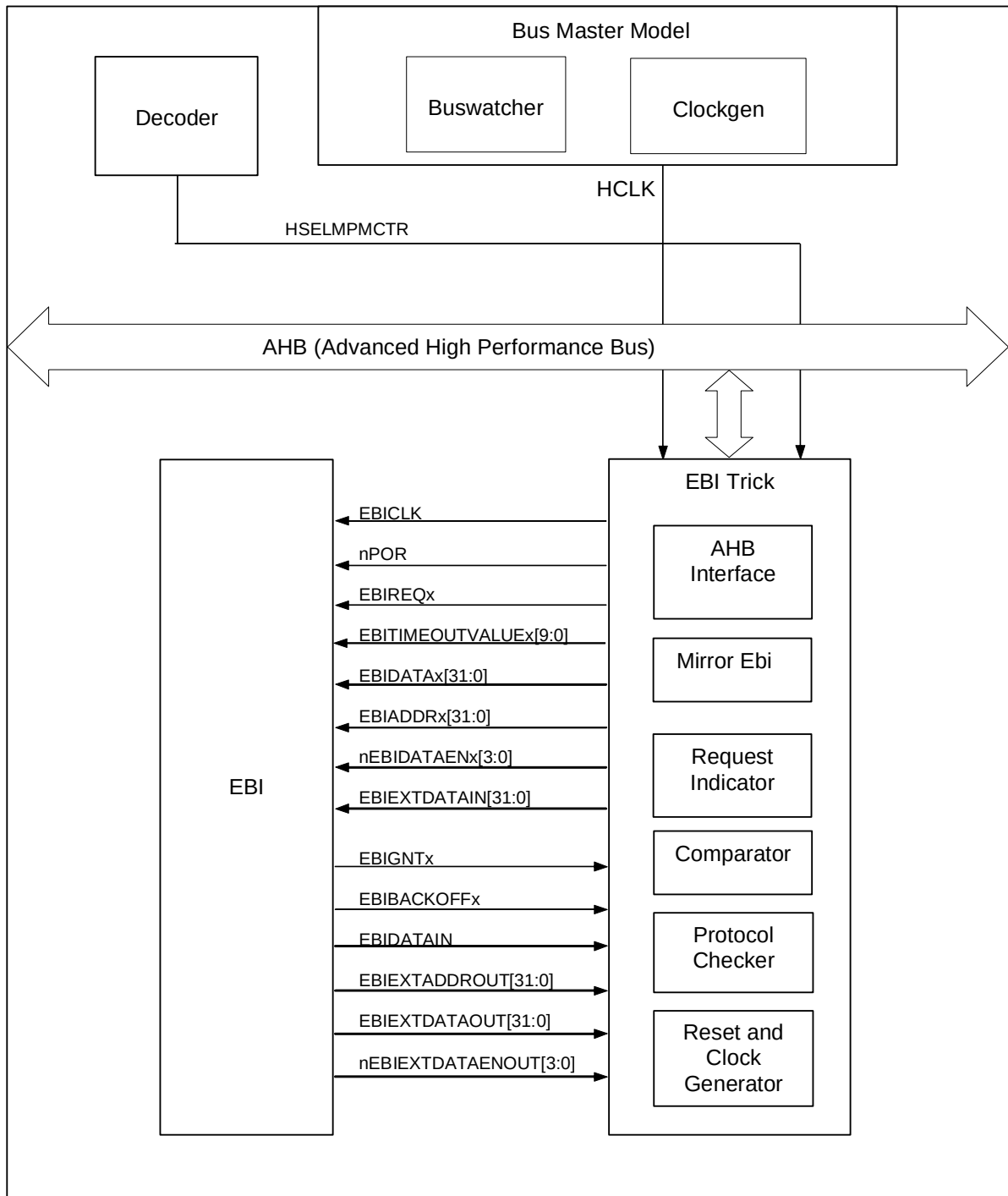


Figure 6.1-1 VHDL Functional Verification Testbench

Figure 6.1-2 illustrates the hierarchy for the VHDL testbench.

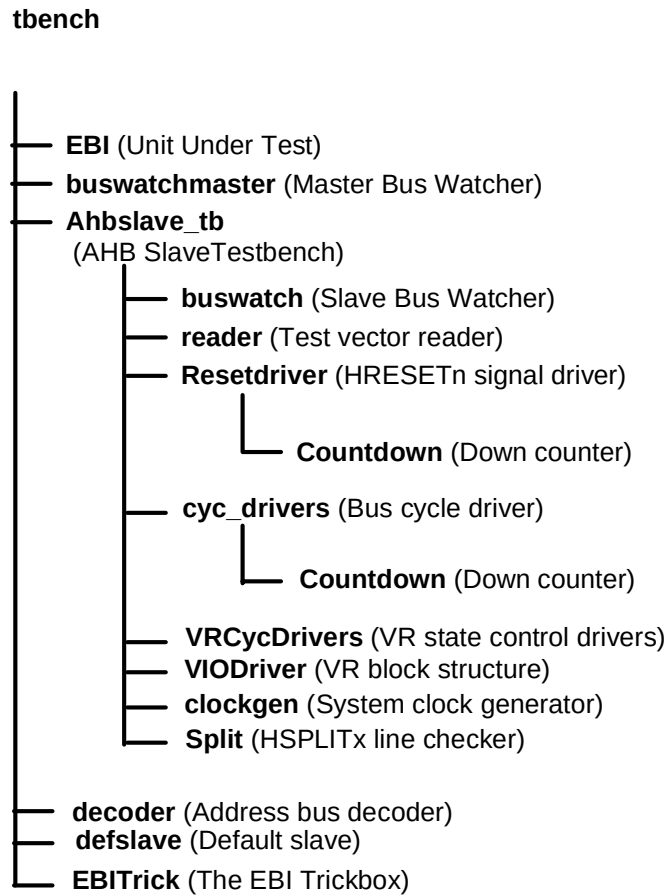


Figure 6.1-2 VHDL Functional Verification Testbench hierarchy

6.1.1 Testbenches

The testbench, **tbench.vhd** is used for functional testing of the External Bus Interface (EBI). The testbench block instantiates the EBI (Unit Under Test) and the trickbox. The behavioural models of the decoder, the default slave and the master bus watcher are done in the testbench.

All Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.vhd file.

6.1.2 Unit Under Test

The Unit Under Test may be one of the following

- EBI VHDL RTL
- EBI scan-inserted VHDL netlist from VHDL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.1.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.vhd` that checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

6.1.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by the AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

There is one AHB master and hence one AHB Master/Arbiter model, `ahbslave_tb.vhd` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the `.bif` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the `HMASTER` number is asserted on the `HSPLIT` lines on the expected clock. The clockgen module generates the system clock, `HCLK`.

The reader reads test vectors from the `infile.bif` residing in the ***verification/bustest/invec*** directory.

6.1.5 Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 AHB slaves.

Any access to the address range

`0x80000000 to 0xBFFFFFFF`

will access the Trickbox via the `HSEL[8]` select line.

Accesses to any address outside the above ranges will select the Default Slave.

6.1.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its `HREADY` output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.1.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.vhd` file in the ***verification/vhdl/tbench*** directory. The buswatch module residing in the ***verification/vhdl/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

6.1.8 Trickbox

The functionality of the EBI is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the input terminals of the EBI and sample its outputs. It samples the output lines from the EBI and checks for the protocols. It flags error messages for any unexpected behaviour detected on the EBI memory signals.

6.1.9 Simulation environment

Generics

The EBI testbench provides some configuration options using generics. All these generics except SuppressOnReset are configurable through the `tbench.vhd` file. SuppressOnReset is configurable through the `ahbslave_tb.vhd` file in the ***verification/vhdl/tbench*** directory.

- XonSig

XonSig is used in the `cyc_drivers.vhd` file that resides in ***verification/vhdl/bid***.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals go to 'X' when the line driver is idle.

- Verbosity

Verbosity is used in the `reset_driver.vhd` and the `cyc_drivers.vhd` which reside in ***verification/vhdl/bid***. It is also used in the `buswatch.vhd` file that resides in ***verification/vhdl/buswatcher*** and the `VIODrivers.vhd` which reside in ***verification/vhdl/vr***.

This generic is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the `cyc_drivers.vhd`, which resides in ***verification/vhdl/bid***, the `buswatch.vhd` which resides in ***verification/vhdl/buswatcher*** and the `VIODriver.vhd` which resides in ***verification/vhdl/vr***.

This generic is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged, but the simulation continues.

- Databuswidth

Databuswidth is used in the `cyc_drivers.vhd`, which resides in ***verification/vhdl/bid***.

This generic controls the width of the AHB databus. For the EBI testbench this is set to 32.

- SuppressOnReset

SuppressOnReset is used in the `buswatch.vhd`, which resides in ***verification/vhdl/buswatcher***.

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

Test Vectors

The test vectors for the verification of the EBI are coded into separate C files, in the ***verification/bustest*** directory. There are separate test files created for each category tests described in the verification plan.

Running Simulations

The **README** file in the ***verification/vhdl/tbench*** directory gives step by step instructions for running simulations using the EBI test vectors on the VHDL source code.

Run time logs for all the combinations are available in the ***verification/vhdl/Ebi_rtl*** and ***verification/vhdl/Ebi_scaninsert_vhdl_net_max*** directories.

6.2 EBI Verilog Verification Testbench

The **Figure 6.2-1** shows the test environment of EBI. The Trickbox is connected to AHB bus. The slave Testbench is responsible for programming the slave registers in Trickbox. The Testbench is interfaced to the Trickbox via AHB Bus. The UUT is interfaced with Trickbox through non-AMBA signals. The Trickbox also does all the protocol checking and drives the non-AMBA signals to the EBI.

Figure 6.2-1 illustrates the EBI Verilog testbench for simulating test vectors.

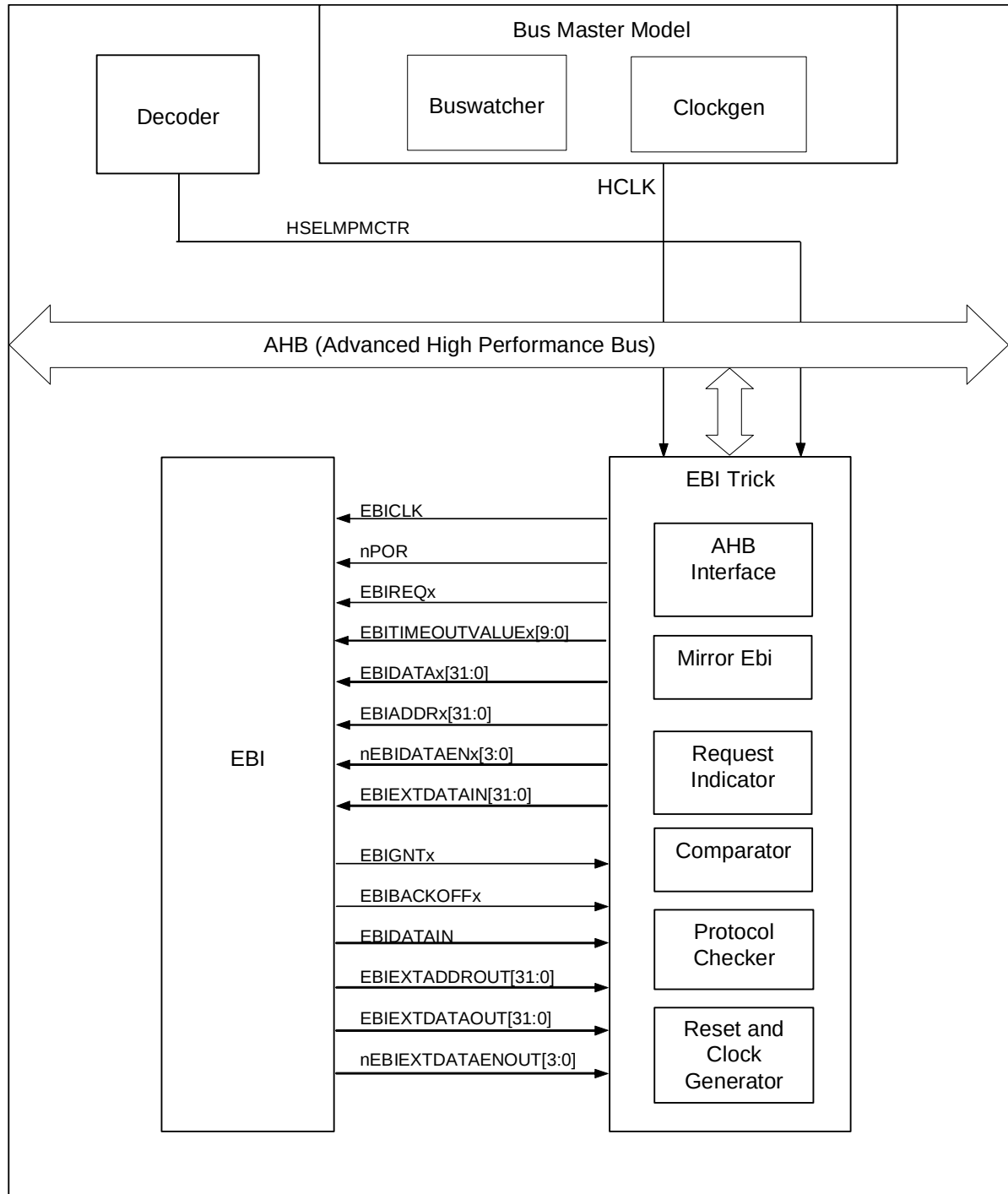


Figure 6.2-1 Verilog Functional Verification testbench

Figure 6.2-2 illustrates the hierarchy for the Verilog testbench.

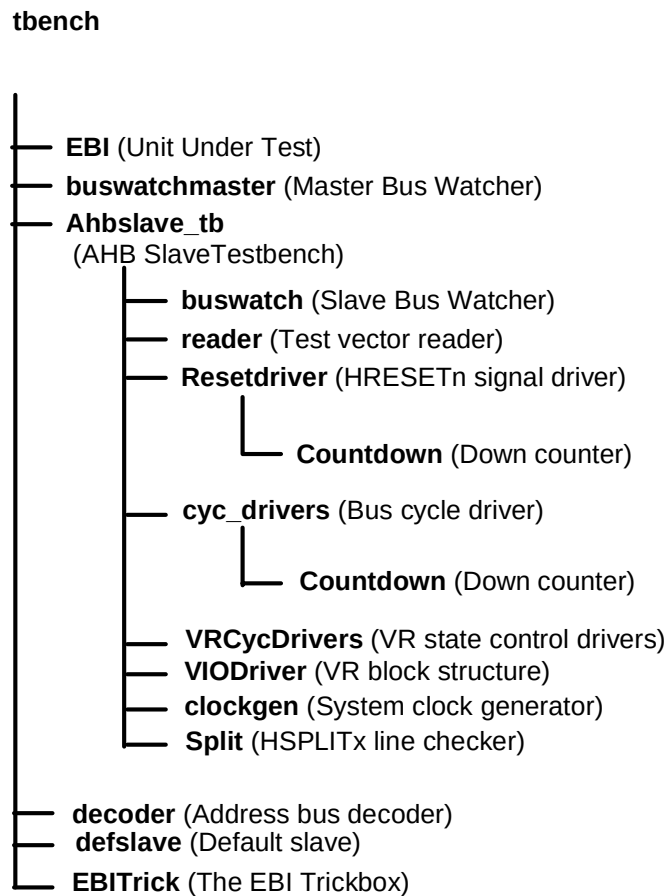


Figure 6.2-2 Verilog Functional Verification Testbench hierarchy

6.2.1 Testbenches

The testbench, **tbench.v** is used for functional testing of the External Bus Interface (EBI). The testbench block instantiates the EBI (Unit Under Test), the trickbox. The behavioural models of the decoder, the default slave and the master bus watcher are done in the testbench.

All Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.v file.

6.2.2 Unit Under Test

The Unit Under Test may be one of the following,

EBI Verilog RTL

EBI scan-ready Verilog netlist from Verilog Source

EBI non-scan Verilog netlist from VHDL Source

One out of these three versions may be referenced by creating a link [named **uut**] in the **verification/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

6.2.3 Bus Master Protocol Checker

The testbench instantiates the master bus watcher, `buswatchmaster.v`, which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and flags warning and error messages if any timing or protocol violations are detected during simulations.

6.2.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

There is one AHB master and hence one AHB Master/Arbiter model, `ahbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the `.sim` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The split module verifies that the `HMASTER` number is asserted on the `HSPLIT` lines on the expected clock. The Clockgen module generates the system clock, `HCLK`.

The reader reads test vectors from the `bif.sim` residing in the ***verification/bustest/invec*** directory.

6.2.5 Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

AHB

Any access to the address range

`0x80000000 to 0xBFFFFFFF`

will access the Trickbox via the `HSEL[8]` select line.

Accesses to any address outside the above ranges will select the Default Slave.

6.2.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its `HREADY` output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

6.2.7 Protocol Checker

The testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.v` file in the ***verification/verilog/tbench*** directory. The buswatch module residing in the ***verification/verilog/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

6.2.8 Trickbox

The functionality of the EBI is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the input terminals of the EBI and sample its outputs. It samples the output lines from the EBI and checks for the protocols. It flags error messages for any unexpected behaviour detected on the EBI memory signals.

6.2.9 Simulation Environment

Simulation configuration (Parameters)

The EBI testbench provides some configurability options via parameters. All the parameters are configurable through the `tbench.v` file.

- XonSig

XonSig is used in the `cyc_drivers.v` file that resides in ***verification/verilog/bid***.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals go to 'X' when the line driver is idle.

- Verbosity

Verbosity is used in the `reset_driver.v` and the `cyc_drivers.v` which reside in ***verification/verilog/bid***. It is also used in the `buswatch.v` that resides in ***verification/verilog/buswatcher*** and the `VIODrivers.v` which reside in ***verification/verilog/vr***.

This parameter is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

- HaltOnMismatch

This is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***, the `buswatch.v` which resides in ***verification/verilog/buswatcher*** and the `VIODriver.v` which resides in ***verification/verilog/vr***.

This parameter is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

- SuppressOnReset

SuppressOnReset is used in the `buswatch.v`, which resides in ***verification/verilog/buswatcher***.

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

- TestMode

TestMode is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

If the TestMode parameter is set, the testbench is configured to run in Big Endian mode. By default this parameter is cleared.

- Databuswidth

Databuswidth is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

This parameter controls the width of the AHB databus. For the EBI testbench this is set to 32.

Test Vectors

The test vectors for the verification of the EBI are coded into separate C files, in the ***verification/bustest*** directory. There are separate test files created for each category tests described in the verification plan. Make options have been provided to use them individually.

Running Simulations

The **README** file in the **verification/verilog/tbench** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the **verification/verilog/Ebi_rtl**, **verification/verilog/Ebi_noscan_vhdl_net_min** and **verification/verilog/Ebi_scanready_verilog_net_typ** directories.

6.3 Trickbox Description

The Trickbox is a programmable AHB slave, designed to drive stimulus on the input terminals of the External Bus Interface and to sample the outputs of the same. The Trickbox also provides a method of validating the EBI.

The Trickbox provides an AHB interface for accessing the internal registers and controlling the data transfer.

The basic blocks of the trickbox are illustrated in **Figure 6.3-1 Block diagram of the trickbox**.

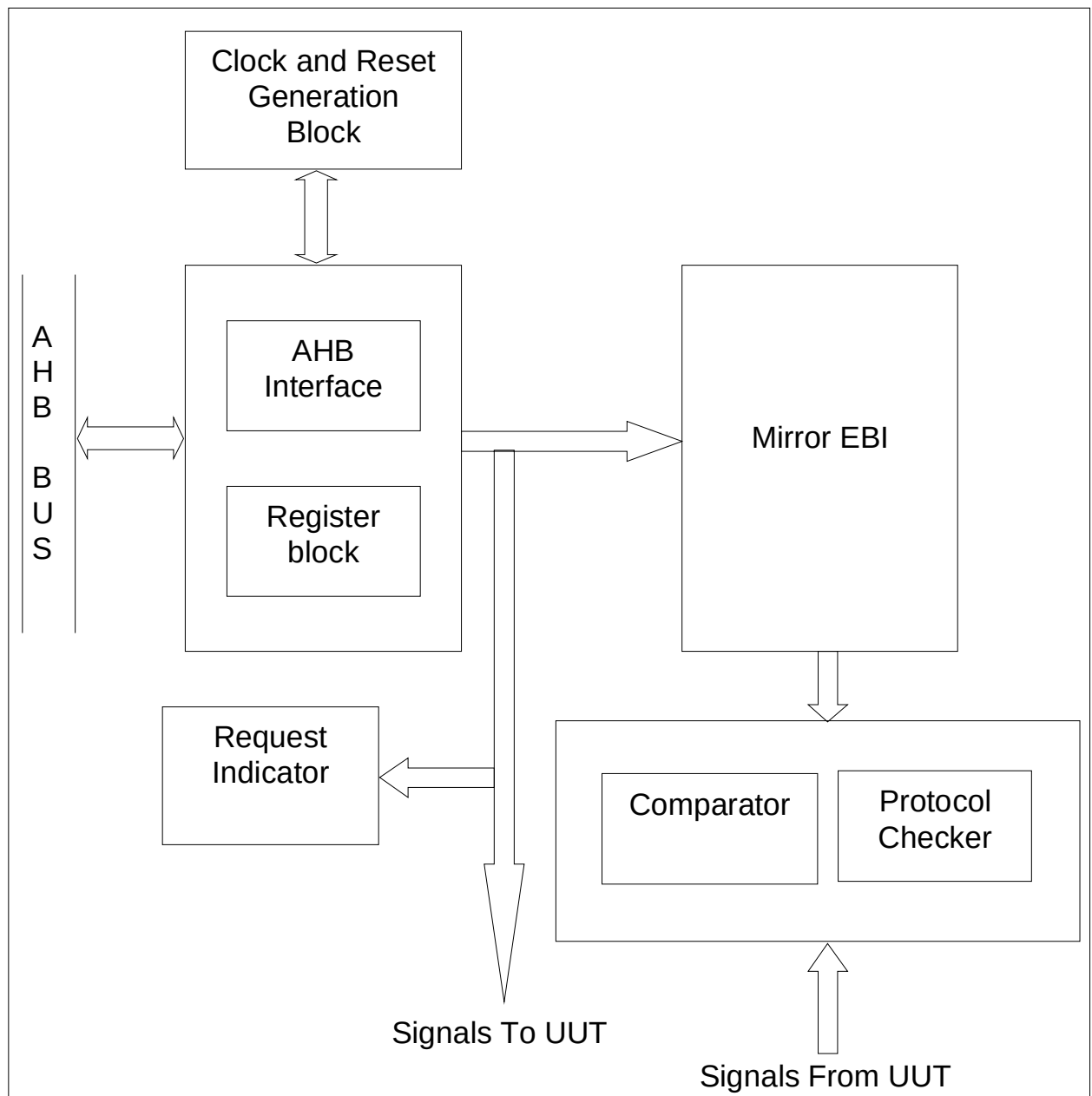


Figure 6.3-1 Block diagram of the trickbox

6.3.1 Trickbox Signal Description

External Bus Interface behavioural TrickBox signals are listed in **Table 6.3-1** and **Table 6.3-2**

Name	Type	Source / Destination	Description
HCLK	IN	From AHB master	AHB clock

Name	Type	Source / Destination	Description
HRESETn	IN	From AHB master	Bus reset signal, Active Low
HADDR[11:2]	IN	From AHB master	Subset of AHB address bus
HTRANS[1:0]	IN	From AHB master	Indicates the type of the current transfer. '00' represents IDLE. '01' represents BUSY. '10' represents NSEQ. '11' represents SEQ.
HSELEBITRICKBOX	IN	From AHB master	Trickbox select signal from decoder. When set to 1 this signal indicates the Trickbox is selected and that a data transfer is required
HSIZE[2:0]	IN	From AHB master	Indicates the size of the transfer to the Trickbox registers. '010' represents 32-bit. If HSIZE[2:0] indicates a transfer width other than 32-bits, the Trickbox returns an ERROR response on HRESP[1:0]
HWRITE	IN	From AHB master	AHB transfer direction signal, indicates a write access when HIGH, read access when LOW
HREADYIN	IN	From AHB master	AHB Transfer Completion Input signal
HWDATA[31:0]	IN	From AHB master	AHB Write databus
HRESP[1:0]	OUT	From AHB master	The transfer response provides additional information on the status of a transfer. Defined responses from AHB slave are, OKAY, ERROR, RETRY and SPLIT. But the EBI trickbox supports only OKAY or ERROR response.
HREADYOUT	OUT	From AHB master	AHB Slave Ready Response
HRDATA[31:0]	OUT	From AHB master	Output Databus to AHB

Table 6.3-1 AHB signal descriptions

Name	Type	Source/Destination	Description
EBICLK	OUT	Clock source	EBI clock
nPOR	OUT	Reset Controller	Power on reset (active low).
EBIREQ1	OUT	To the EBI Port 1	EBI request for port 1. Active high
EBIGNT1	IN	From the EBI	EBI grant for port 1. Active high
EBIBACKOFF1	IN	From the EBI	EBI backoff signal for port 1. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR1[31:0]	OUT	To the EBI Port 1	EBI address for port 1.
EBIDATA1[31:0]	OUT	To the EBI Port 1	EBI data for port 1
nEBIDATAEN1[3:0]	OUT	To the EBI Port 1	Data enable for port 1.
EBITIMEOUTVALUE1[9:0]	OUT	To the EBI Port 1	Gives the value to be loaded into timeout counter for port 1. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ2	OUT	To the EBI	EBI request for port 2. Active high
EBIGNT2	IN	To the EBI	EBI grant for port 2. Active high
EBIBACKOFF2	IN	From the EBI Port 2	EBI backoff for signal port 2. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR2[31:0]	OUT	To the EBI Port 2	EBI address for port 2.
EBIDATA2[31:0]	OUT	To the EBI Port 2	EBI data for port 2
nEBIDATAEN2[3:0]	OUT	To the EBI Port 2	Data enable for port 2.
EBITIMEOUTVALUE2[9:0]	OUT	To the EBI Port 2	Gives the value to be loaded into timeout counter for port 2. Can be tied off to a fixed value or supplied by the system controller.
EBIREQ3	OUT	To the EBI Port 3	EBI request for port 3. Active high
EBIGNT3	IN	From the EBI	EBI grant for port the 3. Active high

Name	Type	Source/Destination	Description
EBIBACKOFF3	IN	From the EBI	EBI backoff signal for port 3. Active high. Indicates to the controller that the current transfer should be completed as soon as possible.
EBIADDR3[31:0]	OUT	To the EBI Port 3	EBI address for port 3.
EBIDATA3[31:0]	OUT	To the EBI Port 3	EBI data for port 3
nEBIDATAEN3[3:0]	OUT	To the EBI Port 3	Data enable for port 3.
EBITIMEOUTVALUE3[9:0]	OUT	To the EBI Port 3	Gives the value to be loaded into timeout counter for port 3. Can be tied off to a fixed value or supplied by the system controller.
EBIDATAIN[31:0]	IN	From the EBI Port 3	Datoin connected to all the memory controllers
EBIEXTADDR[31:0]	IN	From the EBI	Address output to pads
EBIEXTDATAOUT[31:0]	IN	From the EBI	Data output to data pads
nEBIEXTDATAEN[3:0]	IN	From the EBI	Data enable to data pads
EBIEXTDATAIN[31:0]	OUT	To the EBI	Data input from data pads

Table 6.3-2 EBI Interface signal description

6.3.2 AHB Interface Block

This block interfaces the EBI Trickbox with the AHB. The Trickbox is an AHB slave and it supports read-write operations of transfer size 32-bits and burst type of INCR. The AHB interface block performs the following operations.

- AHB response generation logic to generate the HREADYOUT and HRESP[1:0] signals
- Error monitoring to detect the transfers that result in an error response.
- Generation of read and write decodes for all registers.

A block diagram of the AHB Interface block is in **Figure 6.3-2**

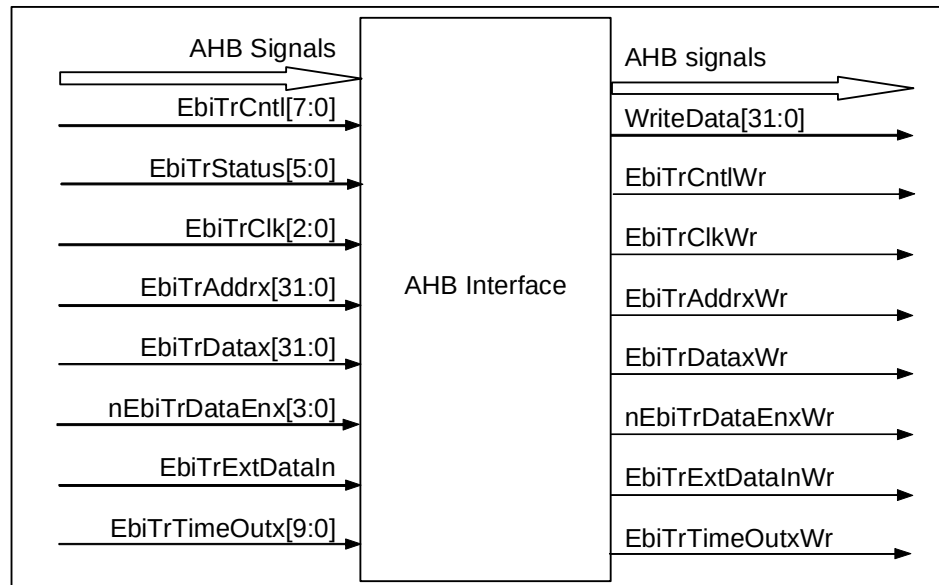


Figure 6.3-2 Block diagram of AHB interface

6.3.3 Register Block

This block implements the AHB read-write registers used in the Trickbox. This block generates the input signal for the EBI by programming the registers value. A block diagram of the AHB Interface block is in **Figure 6.3-3**

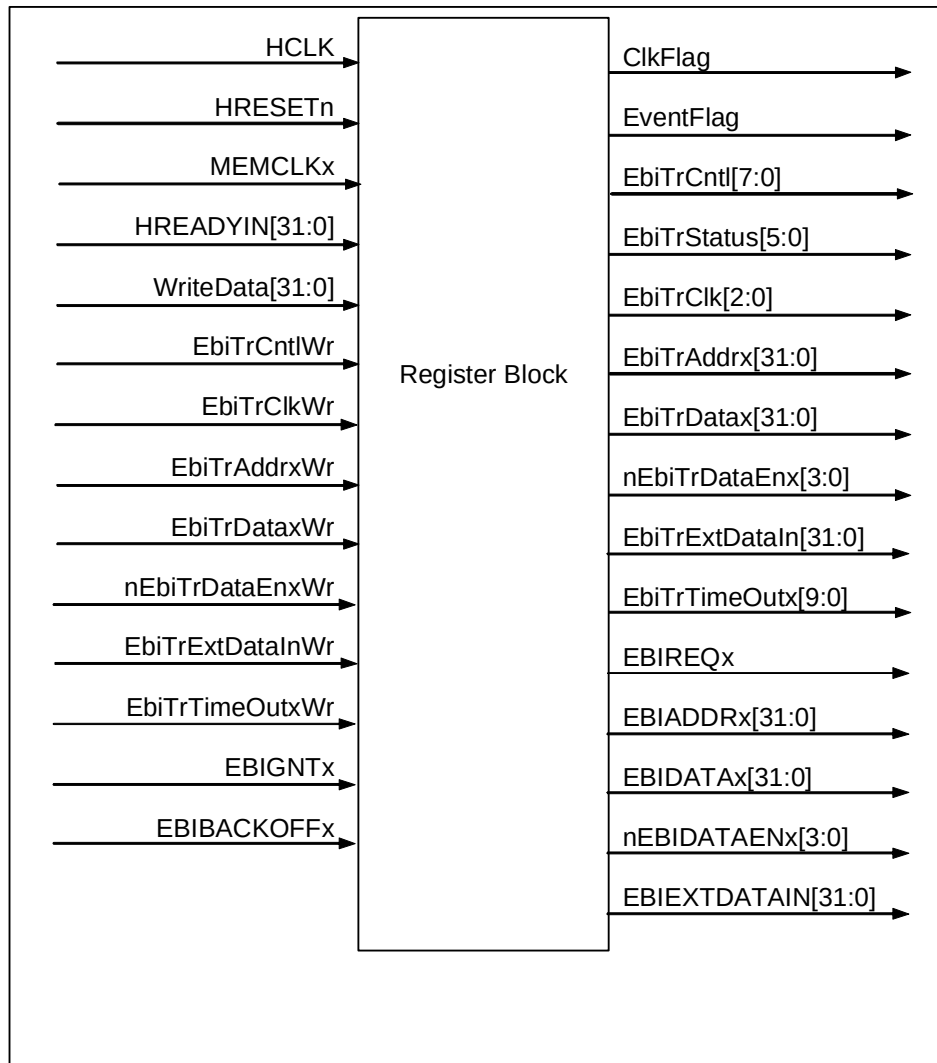


Figure 6.3-3 Block diagram of Register Block

The EBI Trickbox registers are shown in **Table 6.3-3** and the **Table 6.3-4** shows the signal name and the clocks used to generate the signal. Each register's description is given in the heading **Trickbox register summary**

The Base address of the trickbox is not fixed and depends on the system implementation. However, the offset of any particular register from the Base address is fixed.

Address	Type	Width	Reset Value	Register Name	Description
EBI Trick base + 0x0000	R/W	7-0	0x0	EbiTrCntl	Controls the EBI request and other signals
EBI Trick base + 0x0004	R	5-0	0x0	EbiTrStatus	Gives the status of the EBIGNTx and EBIBACKOFFx signals
EBI Trick base + 0x0008	R/W	2-0	0x0	EbiTrClk	Controls the MEMCLKx speed

Address	Type	Width	Reset Value	Register Name	Description
EBI Trick base + 0x000C	R/W	31-0	0x0	EbiTrAddr1	Gives the Address value to be put on the EBIADDR1 bus
EBI Trick base + 0x0010	R/W	31-0	0x0	EbiTrAddr2	Gives the Address value to be put on the EBIADDR2 bus
EBI Trick base + 0x0014	R/W	31-0	0x0	EbiTrAddr3	Gives the Address value to be put on the EBIADDR3 bus
EBI Trick base + 0x0018	R/W	31-0	0x0	EbiTrData1	Gives the Data value to be put on the EBIDATA1 bus
EBI Trick base + 0x001C	R/W	31-0	0x0	EbiTrData2	Gives the Data value to be put on the EBIDATA2 bus
EBI Trick base + 0x0020	R/W	31-0	0x0	EbiTrData3	Gives the Data value to be put on the EBIDATA3 bus
EBI Trick base + 0x0024	R/W	3-0	0x0	nEbiTrDataEn1	Gives the Data enable value to be put on the nEBIDATAEN1 bus
EBI Trick base + 0x0028	R/W	3-0	0x0	nEbiTrDataEn2	Gives the Data enable value to be put on the nEBIDATAEN2 bus
EBI Trick base + 0x002C	R/W	3-0	0x0	nEbiTrDataEn3	Gives the Data enable value to be put on the nEBIDATAEN3 bus
EBI Trick base + 0x0030	R/W	31-0	0x0	EbiTrExtDataIn	Gives the External Data In value to be put on the EBIEXTDATAIN bus
EBI Trick base + 0x0034	R/W	9-0	0x0	EbiTrTimeOut1	Gives the timeout value to be put on EBITIMEOUTVALUE1 bus
EBI Trick base + 0x0038	R/W	9-0	0x0	EbiTrTimeOut2	Gives the timeout value to be put on EBITIMEOUTVALUE2 bus
EBI Trick base + 0x003C	R/W	9-0	0x0	EbiTrTimeOut3	Gives the timeout value to be put on EBITIMEOUTVALUE3 bus

Table 6.3-3 Trickbox register

S.No	Signal	CLOCK
1	EBIREQ1	MEMCLK1
2	EBIDATA1	MEMCLK1
3	EBIADDR1	MEMCLK1
4	nEBIDATAEN1	MEMCLK1
5	EBITIMEOUTVALUE1	MEMCLK1
6	EBIREQ2	MEMCLK2
7	EBIDATA2	MEMCLK2
8	EBIADDR2	MEMCLK2
9	nEBIDATAEN2	MEMCLK2
10	EBITIMEOUTVALUE2	MEMCLK2

11	EBIREQ3	MEMCLK3
12	EBIDATA3	MEMCLK3
13	EBIADDR3	MEMCLK3
14	nEBIDATAEN3	MEMCLK3
15	EBITIMEOUTVALUE3	MEMCLK3
16	EBIEXTDATAIN	HCLK

Table 6.3-4 EBI Input signals and clocks

6.3.4 Clock and Reset Generator

This block performs the following functions:

- Generates **MEMCLK1**, **MEMCLK2**, **MEMCLK3**, and **EBICLK**.
- Generates **nPOR**

A block diagram of the Clock and Reset Generation block is in **Figure 6.3-4**

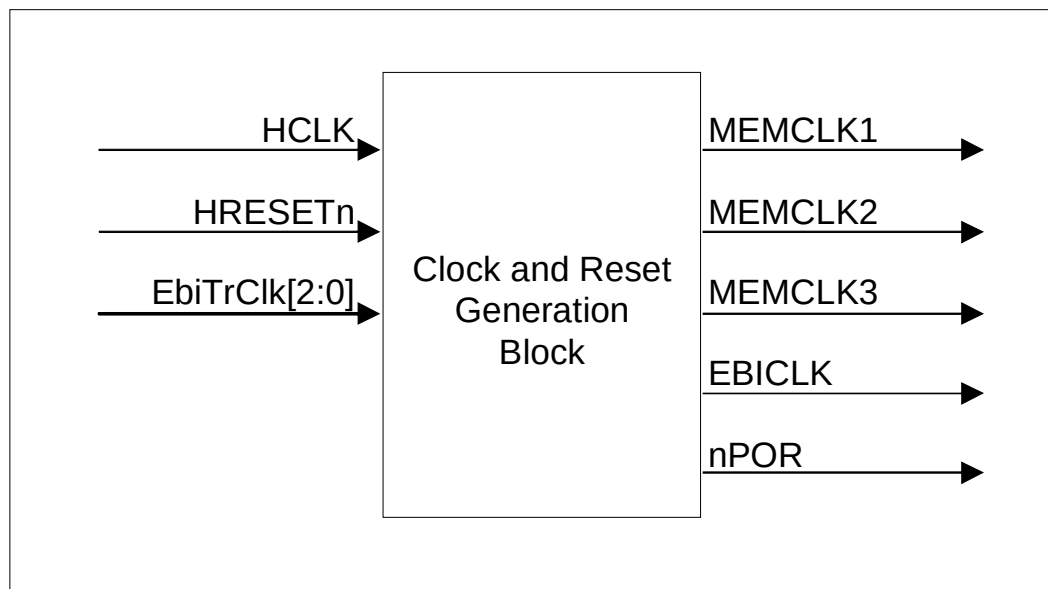


Figure 6.3-4 Block diagram of the Clock and Reset Generation block

The signal **EbiTrClk[2:0]** determines the speed of the clocks. The **MEMCLKx** speed may be the equal to **HCLK** or double of the **HCLK** depending on the **EbiTrClk[2:0]**. The frequency of **EBICLK** is equal to the maximum frequency of the **MEMCLKx**. The signal **nPOR** is generated from the **HRESETn**.

6.3.5 EBI Mirror Trick Box

The trickbox has a behavioural model EBI Mirror Trick box, which is used to check the functionality of the EBI model. The same input stimuli are applied to both the EBI model and the EBI Mirror Trick box model. The output from the EBI Mirror Trick box and the EBI model are compared on a clock-by-clock basis by the comparator. EBI Mirror Trick generates the same outputs as EBI Model. The **Figure 6.3-5** shows the input and outputs of EBI Mirror Trick box model.

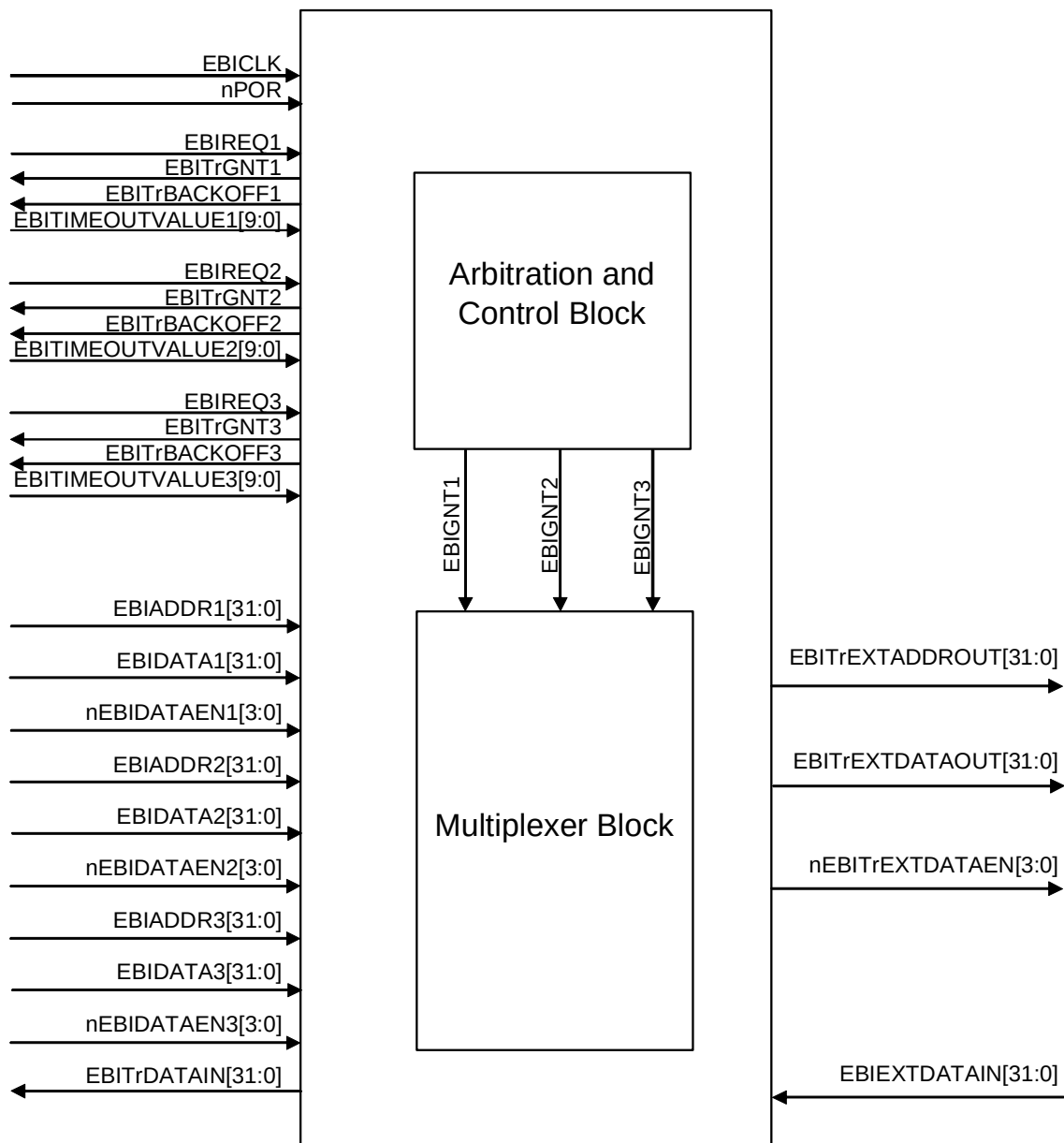


Figure 6.3-5 Top-Level Diagram of EBI Mirror Trick Box

The top-level diagram of the EBI Mirror Trick box illustrates the Input and Output signals. The EBI Mirror Trick box is internally implemented as two sub-Blocks.

Multiplexer Block (EbiTrMultBlk)

This block implements the multiplexers for the address, data and data enable signals. The Multiplexer block multiplexes the address, data and data enable lines from three separate Controllers on to the common address, data and data enable pins of the chip based on the granted port.

Arbitration and Control Block (EbiTrArbtCtl)

This block implements the Arbitration and Control block for the Ebi Mirror Trick Box. The Arbitration Block and Control block in Ebi Mirror Trick Box arbitrates between the requested ports. Each port has a 10-bit counter, which will be loaded with a value from EBITimeOut [9:0] input whenever a request is made for the bus. If the signal EBITimeOut [9:0] is zero then the counter will not be started for the port. When the counter reaches zero then port which is

currently granted the bus, will be requested to release the bus by asserting the Ebibackoff signal. The first port counter to reach zero will be given the highest priority for the bus, followed by the second device. If more than one port issues a request for the bus at the same time then a round robin arbitration scheme will be used to decide which port to be granted the bus. When no device is requesting than the most recently granted port would continue to be granted the bus. At reset port one will be granted the bus.

The Arbitration and Control block is implemented as a single state machine. This state machine makes State transitions based on the requests from Port 1, Port 2 and Port 3. At every positive clock edge the requests from all the three ports are sampled and depending upon the requests

State transitions are made. State Transition Diagram is shown in **Figure 6.3-6**

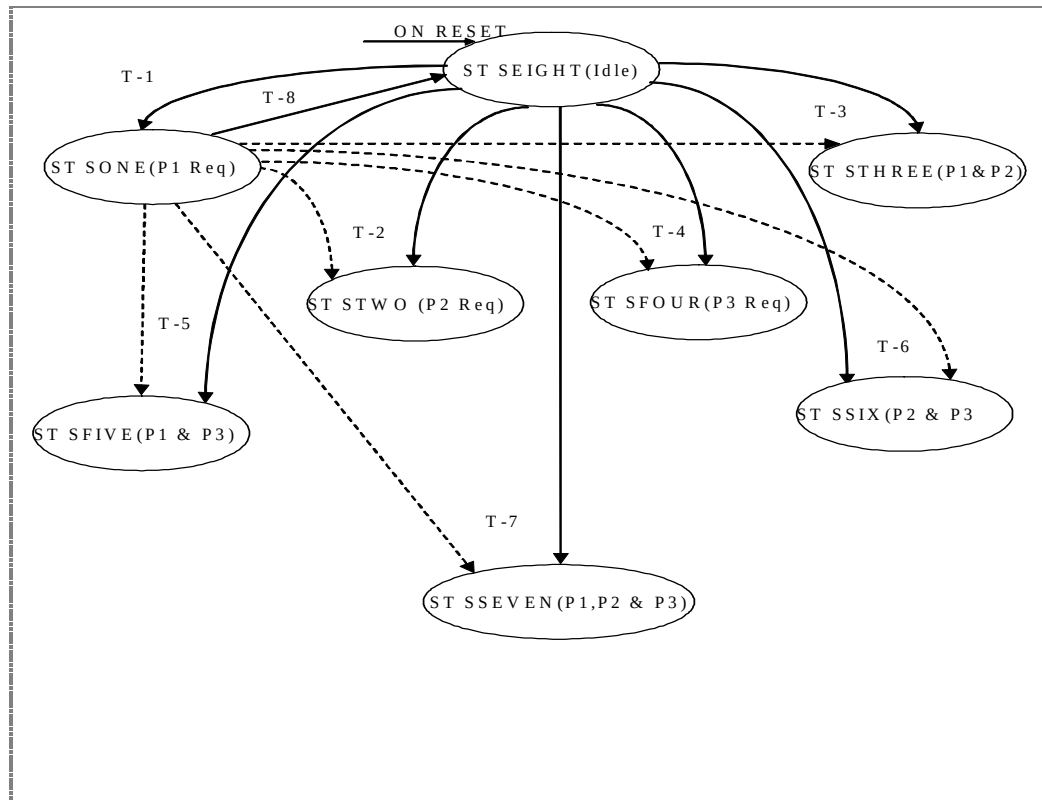


Figure 6.3-6 Request State Machine

The Request State Machine transitions through following states:

ST_SEIGHT: Idle State when there is no request from Port 1, Port 2 or Port 3.

ST_SONE: When there is a request from Port 1 only.

ST_STWO: When there is a request from Port 2 only.

ST_STHREE: When there is a request from Port 1 and Port 2. The request could have been asserted in any order, like Port 1 first and Port 2 later or vice versa. The transition to this state is also made when there is simultaneous request from Port 1 and Port 2.

ST_SFOUR: When there is a request from Port 3 only.

ST_SFIVE: When there is a request from Port 1 and Port 3. The request could have been asserted in any order, like Port 3 first and Port 1 later or vice versa. The transition to this state is also made when there are simultaneous requests from Port 3 and Port 1.

ST_SSEX: When there is a request from Port 2 and Port 3. The request could have been asserted in any order, like Port 3 first and Port 2 later or vice versa. The transition to this state is also made when there are simultaneous requests from Port 3 and Port 2.

ST_SSEVEN: When all three Ports assert the request this transition is made. The order of assertion of request can be in any order. The transition to this state is also made when there are simultaneous requests from Port 1, Port 2 and Port 3.

In **Table 6.3-5**, transitions from ST_SEIGHT and ST_SONE to each of the seven other states are shown. Similarly from every state transition to all other states are made. This has not been shown in the figure for reasons of clarity.

State Transitions Table

Transition	Description	Port3 Request	Port2 Request	Port1 Request
T1	Transition to state ST_SONE from any other state is made when only Port 1 request is present.	0	0	1
T2	Transition to state ST_STWO from any other state is made when only Port 2 request is present	0	1	0
T3	Transition to state ST_STHREE from any other state is made when only Port1 and Port 2 request is present	0	1	1
T4	Transition to state ST_SFIVE from any other state is made when only Port 1 request is present	1	0	0
T5	Transition to state ST_SFIVE from any other state is made when only Port3 and Port 1 request is present	1	0	1
T6	Transition to state ST_SSIX from any other state is made when only Port3 and Port 2 request is present	1	1	0
T7	Transition to state ST_SSEVEN from any other state is made when all the Port requests are present	1	1	1
T8	Transition to state ST_SEIGHT from any other state is made when all the requests are deasserted or	0	0	0

	when no requests are present			
--	---------------------------------	--	--	--

Table 6.3-5 State Transition Table

The transitions to states ST_STHREE, ST_SFIVE, ST_SSEVEN and ST_SSIX could be due to simultaneous requests from respective ports or one after the other in any sequence. ST_SEIGHT is an idle state when there is no request. In this state, the previous granted port will continue to be granted. In each state eight transitions are possible. A Priority Table is used to resolve priority between two ports. Three bits are used to represent Port Id.

Priority Table

The Priority table is a structure which has two locations each of three bits wide. Three bits are used to represent the Port number as shown below. When a port is granted the port number is written at location zero and the contents of location zero is pushed to location one. In this manner the priority table is updated and when a priority needs to be resolved, the contents of these locations are checked to know the least granted port.

The three bits port number is shown below:

Port number

001 -> Port 1

010 -> Port 2

100 -> Port 3

BackoffReg

Backoff register is two bits wide. To update Backoff register two bits are used to represent port number as shown below. This Backoff register contains a port number whose time out counter has reached zero first. This helps in generating the grant signal when more than one counter reaches time out value zero.

The two bits port number is shown below:

00 -> Cleared

01 -> Port 1

10 -> Port 2

11 -> Port 3

The grant generation mechanism works using the Priority Table and Backoff Register and requests from Port 1, Port 2 and Port 3. Outputs of the EBI Mirror Trick box is used by the comparator for comparison with the outputs from the EBI Model.

6.3.6 Comparator

This block is responsible for doing the entire clock by clock comparisons of the UUT output with the expected value, which is getting generated in the Mirror EBI. Whenever it finds a mismatch between the signals from UUT and the signals from the Mirror EBI, it displays error messages

A block diagram of the comparator block is in **Figure 6.3-7**. The list of signals, which gets compared with the UUT, is given below in **Table 6.3-6**.

Serial No.	Signal Name
1	EBIGNT1
2	EBIGNT2
3	EBIGNT3
4	EBIBACKOFF1
5	EBIBACKOFF2

6	EBIBACKOFF3
7	EBIEXTDATAOUT
8	EBIEXTADDRROUT
9	nEBIEXTDATAEN
10	EBIDATAIN

Table 6.3-6 List of signals compared in the comparator block

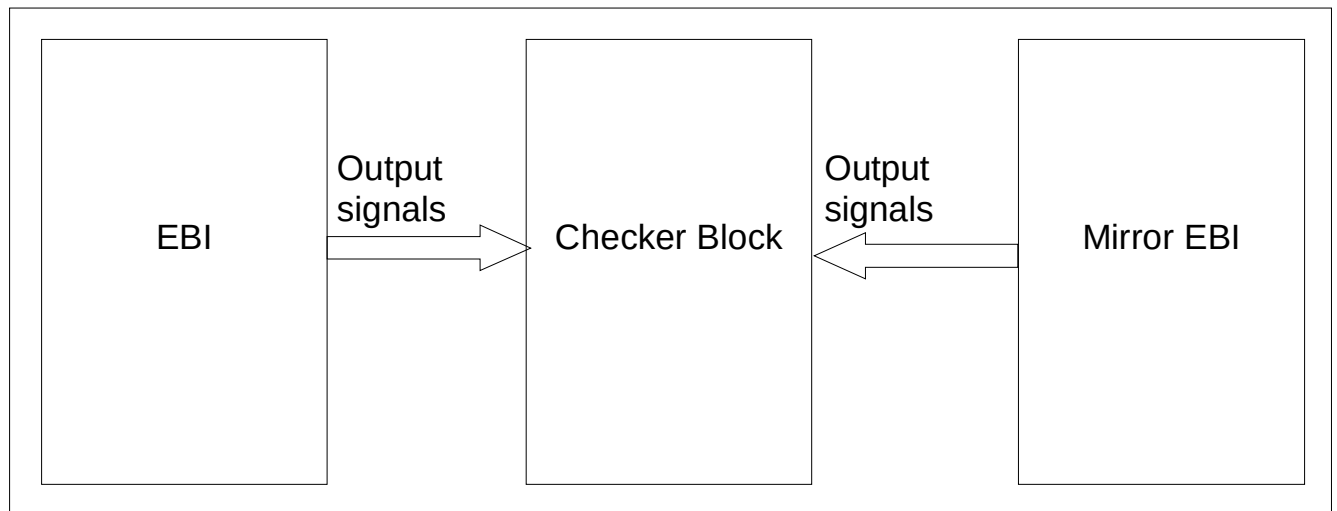


Figure 6.3-7 Block diagram of the comparator module

6.3.7 Protocol checker

It performs the following functionality:

- Checks for 'X'es on the EBI output signals.
- Checks for multiple assertions of Grant lines.
- Checks for multiple assertions of Backoff signals.

Whenever a violation is found in the EBI protocol, the Protocol Checker Block flags error messages.

6.3.8 EBI Request Indicator

This block gives details of the assertion of the signals **EBIREQ1**, **EBIREQ2** and **EBIREQ3** with respect to **EBICLK** and Request event. It helps to identify if the required test condition is hit or not. The **EBIREQx** signals are checked clock by clock if the **ClkFlag** is set and the following messages are displayed with respect to the condition of the **EBIREQx** signals

- No change in the value of EBIREQ1 since last clock
- EBIREQ1 has become high from low
- EBIREQ1 has become low from high
- No change in the value of EBIREQ2 since last clock

- EBIREQ2 has become high from low
- EBIREQ2 has become low from high
- No change in the value of EBIREQ3 since last clock
- EBIREQ3 has become high from low
- EBIREQ3 has become low from high

The **EBIREQx** signals are checked on an event basis if the **EventFlag** is set and the following messages are displayed with respect to the condition of the **EBIREQx** signals

- EBIREQ1 has become high from low
- EBIREQ1 has become low from high
- EBIREQ2 has become high from low
- EBIREQ2 has become low from high
- EBIREQ3 has become high from low
- EBIREQ3 has become low from high

6.3.9 Trickbox register summary

EbiTrCntl: Control Register

This is an 8-bit read/write register for controlling **EBIREQx** signal.

Bits	Name	Read/Write	Function
31:8	Reserved	-	
7	EventFlag	R/W	When enabled checks EbiReq lines status when there is an event on any of the EbiReq lines
6	ClkFlag	R/W	When enabled checks EbiReq lines status in each clock
5	EbiReqPos3	R/W	When enabled EbiReq3 will be posted one MemClk3 later
4	EbiReqPos2	R/W	When enabled EbiReq2 will be posted one MemClk2 later
3	EbiReqPos1	R/W	When enabled EbiReq1 will be posted one MemClk1 later
2	EbiTrReq3	R/W	Request from Port3
1	EbiTrReq2	R/W	Request from Port2
0	EbiTrReq1	R/W	Request from Port1

Table 6.3-7 EbiTrCntl bit description

If EbiTrReq1, EbiTrReq2, EbiTrReq3 bits are asserted then corresponding **EBIREQx** lines to EBI are asserted. When these bits are low, corresponding lines are de asserted.

When EbiReqPos1 is high **EBIREQ1** will be posted one **MEMCLK1** later than if this bit is not set.

When EbiReqPos2 is high **EBIREQ2** will be posted one **MEMCLK2** later than if this bit is not set.

When EbiReqPos3 is high **EBIREQ3** will be posted one **MEMCLK3** later than if this bit is not set.

All EbiTrReqx bits of EbiTrCntl register are written in accordance with **HCLK** irrespective of clock speed bits and requests will be raised. If EbiReqPosx is high then, **EBIREQx** lines will be asserted one **MEMCLKx** later.

EbiTrStatus: Status Register

This is a 6-bit read only register for reading the status of EBI.

Bits	Name	Read/Write	Function
31:6	Reserved	-	-
5	EbiTrBackOff3	R	Indicates the EBIBACKOFF3 signal of Port3
4	EbiTrBackOff2	R	Indicates the EBIBACKOFF2 signal of Port2
3	EbiTrBackOff1	R	Indicates the EBIBACKOFF1 signal of Port1
2	EbiTrGnt3	R	Indicates the EBIGNT3 of Port3
1	EbiTrGnt2	R	Indicates the EBIGNT2 of Port2
0	EbiTrGnt1	R	Indicates the EBIGNT1 of Port1

Table 6.3-8 EbiTrStatus bit description

EbiTrGnt1, EbiTrGnt2, EbiTrGnt3 bits indicate the status of **EBIGNT1**, **EBIGNT2** and **EBIGNT3** signal of the EBI.

EbiTrBackOff1, EbiTrBackOff2, EbiTrBackOff3 bits indicate the status of **EBIBACKOFF1**, **EBIBACKOFF2** and **EBIBACKOFF3** signals of the EBI.

EbiTrClk: Clock Register

This is a 3-bit read/write register for controlling the MemClkx timing.

Bits	Name	Read/Write	Function
31:3	Reserved	-	-
2	MemClk3	R/W	0 = MEMCLK3 and HCLK are Equal 1 = MEMCLK3 is the double of HCLK
1	MemClk2	R/W	0 = MEMCLK2 and HCLK are Equal 1 = MEMCLK2 is the double of HCLK
0	MemClk1	R/W	0 = MEMCLK1 and HCLK are Equal 1 = MEMCLK1 is the double of HCLK

Table 6.3-9 EbiTrClk bit description

EbiTrAddr1: Address 1 Register

This is a 32-bit read/write register for controlling the **EBIADDR1** of the EBI.

Bits	Name	Read/Write	Function
31:0	Addr1	R/W	Gives the address to be put on the EBIADDR1 bus

Table 6.3-10 EbiTrAddr1 bit description

EbiTrAddr2: Address 2 Register

This is a 32-bit read/write register for controlling the **EBIADDR2** of the EBI.

Bits	Name	Read/Write	Function
31:0	Addr2	R/W	Gives the address to be put on the EBIADDR2 bus

Table 6.3-11 EbiTrAddr2 bit description**EbiTrAddr3: Address 3 Register**

This is a 32-bit read/write register for controlling the **EBIADDR3** of the EBI.

Bits	Name	Read/Write	Function
31:0	Addr3	R/W	Gives the address to be put on the EBIADDR3 bus

Table 6.3-12 EbiTrAddr3 bit description**EbiTrData1: Data 1 Register**

This is a 32-bit read/write register for controlling the **EBIDATA1** of the EBI.

Bits	Name	Read/Write	Function
31:0	Data1	R/W	Gives the data to be put on the EBIDATA1 bus

Table 6.3-13 EbiTrData1 bit description**EbiTrData2: Data 2 Register**

This is a 32-bit read/write register for controlling the **EBIDATA2** of the EBI.

Bits	Name	Read/Write	Function
31:0	Data2	R/W	Gives the data to be put on the EBIDATA2 bus

Table 6.3-14 EbiTrData2 bit description**EbiTrData3: Data 3 Register**

This is a 32-bit read/write register for controlling the **EBIDATA3** of the EBI.

Bits	Name	Read/Write	Function
31:0	Data3	R/W	Gives the data to be put on the EBIDATA3 bus

Table 6.3-15 EbiTrData3 bit description**nEbiTrDataEn1: Data enable 1 Register**

This is a 4-bit read/write register for controlling the **nEBIDATAEN1** of the EBI.

Bits	Name	Read/Write	Function
31:4	Reserved	-	-
3:0	DataEn1	R/W	Gives the Data enable to be put on the nEBIDATAEN1 bus

Table 6.3-16 nEbiTrDataEn1 bit description**nEbiTrDataEn2: Data enable 2 Register**

This is a 4-bit read/write register for controlling the **nEBIDATAEN2** of the EBI.

Bits	Name	Read/Write	Function
31:4	Reserved	-	-
3:0	DataEn2	R/W	Gives the Data enable to be put on the nEBIDATAEN2 bus

Table 6.3-17 nEbiTrDataEn2 bit description

nEbiTrDataEn3: Data enable 3 Register

This is a 4-bit read/write register for controlling the **nEBIDATAEN3** of the EBI.

Bits	Name	Read/Write	Function
31:4	Reserved	-	-
3:0	DataEn3	R/W	Gives the Data enable to be put on the nEBIDATAEN3 bus

Table 6.3-18 nEbiTrDataEn3 bit description

EbiTrExtDataIn: External Data In Register

This is a 32-bit read/write register for controlling the **EBIEXTDATAIN** of the EBI.

Bits	Name	Read/Write	Function
31:0	ExtDataIn	R/W	Gives the data to be put on the EBIEXTDATAIN bus

Table 6.3-19 EbiTrData3 bit description

EbiTrTimeOut1: Timeout 1 Register

This is a 10-bit read/write register for controlling the **EBITIMEOUTVALUE1** of the EBI.

Bits	Name	Read/Write	Function
31:10	Reserved	-	-
9:0	TimeOut1	R/W	Gives the timeout value to be put on the EBITIMEOUTVALUE1 bus

Table 6.3-20 EbiTrTimeOut1 bit description

EbiTrTimeOut2: Timeout 2 Register

This is a 10-bit read/write register for controlling the **EBITIMEOUTVALUE2** of the EBI.

Bits	Name	Read/Write	Function
31:10	Reserved	-	-
9:0	TimeOut2	R/W	Gives the timeout value to be put on the EBITIMEOUTVALUE2 bus

Table 6.3-21 EbiTrTimeOut2 bit description

EbiTrTimeOut3: Timeout 3 Register

This is a 10-bit read/write register for controlling the **EBITIMEOUTVALUE3** of the EBI.

Bits	Name	Read/Write	Function
------	------	------------	----------

31:10	Reserved	-	-
9:0	TimeOut3	R/W	Gives the timeout value to be put on the EBITIMEOUTVALUE3 bus

Table 6.3-22 EbiTrTimeOut3 bit description

7 VERIFICATION TESTS

7.1 FUNCTIONAL TESTS

7.1.1 EBIMClkEqHCLK Test

In this test case, the Memory clock is equal to HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMClkEqHCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. EbiTrClk register present in the trick box is programmed to zero in order to keep memory clock equal to HCLK. Programming ClkFlag bit of EbiTrCntl register in the trick box high enables the clock flag.

Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are performed when Memory Clocks are kept equal to HCLK

- Request asserted and deasserted from single port. Exercised for all the three ports.
- Requests asserted simultaneously from two ports and deasserted sequentially. Exercised for all three ports.
- Requests raised simultaneously from all the ports.
- Requests from two ports asserted simultaneously and deasserted simultaneously. Exercised for all three ports.
- Requests asserted and deasserted sequentially one after the other.
- Request asserted by a port for a single clock.
- Request asserted by a port for a single clock and another port request is asserted on the same clock edge as that of deassertion of the first port's request.
- Generation of Backoff signal is tested under different conditions.
- Grant mechanism based on Backoff signal is checked.
- Requests are kept higher for longer duration of time.
- Grant mechanism based on least recently used algorithm i.e. the least recently used port gets the highest priority and the most recently used port gets the lowest priority, is checked.

7.1.2 EBIMClk1Eq2HCLK Test

In this test case, the MemClk1 is equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMClk1Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk1 bit of EbiTrClk register present in the trick box is programmed one in order to achieve MemClk1 equal to twice HCLK. Programming EventFlag bit of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks that the following conditions are performed when Memory Clocks is made equal to twice that of HCLK with EbiReqPos1 bit set in the EbiTrCntl register.

- Request asserted and deasserted by Port1.

- Request asserted and deasserted by Port1 and Port2.
- Request asserted and deasserted by Port1 and Port3.
- Request asserted and deasserted by Port2 and Port3.
- Request asserted and deasserted by all the three Ports.
- Request asserted sequentially and deasserted simultaneously by all the three ports.
- Request asserted by Port 1 for one clock cycle.
- Backoff mechanism on the granted port is checked.
- Request for all three ports raised simultaneously, Port 2 and Port 3 timeout values loaded are same, when Port 1 releases the request the least recently used algorithm is checked.

7.1.3 EBIMClk2Eq2HCLK Test

In this test case, the MemClk2 is equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EbiMClk2Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box.

Program some values in the counter register corresponding to each port present in the trick box. MemClk2 bit of EbiTrClk register present in the trick box is programmed in order to achieve MemClk2 equal to twice HCLK. Programming EventFlag bit of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are performed when MemClk2 is made equal to twice that of HCLK with EbiReqPos2 bit set in the EbiTrCntl register.

- Request asserted and deasserted by Port2.
- Request asserted and deasserted by Port1 and Port2.
- Request asserted and deasserted by Port2 and Port3.
- Request asserted and deasserted by all the three Ports.
- Request asserted sequentially and deasserted simultaneously by all the three ports.
- Request asserted by Port 2 for one clock cycle.
- Backoff mechanism on the granted port is checked.
- Request for all three ports raised simultaneously, Port 2 and Port 3 timeout values loaded are same, when Port 1 releases the request the least recently used algorithm is checked.

7.1.4 EBIMClk3Eq2HCLK Test

In this test case, the MemClk3 is equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMClk3Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk3 bit of EbiTrClk register present in the trick box is programmed in order to achieve MemClk3 equal to twice HCLK. Programming EventFlag Bit of EbiTrCntl register in the trick box high enables the event flag.

Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are performed when MemClk3 is equal to twice HCLK with EbiReqPos3 bit set in the EbiTrCntl register.

- Request asserted and deasserted by Port3.
- Request asserted and deasserted by Port1 and Port3.
- Request asserted and deasserted by Port2 and Port3.
- Request asserted and deasserted by Port1 and Port2.
- Request asserted and deasserted by all the three Ports.
- Request asserted sequentially and deasserted simultaneously by all the three ports.
- Request asserted by Port 3 for one clock cycle.
- Backoff mechanism on the granted port is checked.
- Request for all three ports raised simultaneously, Port 1 and Port 3 timeout values loaded are same, when Port 2 releases the request the least recently used algorithm is checked.

7.1.5 EBIMck1Mck2Eq2HCLK Test

In this test case the MemClk1 and MemClk2 is equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMck1Mck2Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk1 and MemClk2 bits of EbiTrClk register present in the trick box is programmed in order to achieve MemClk1 and MemClk2 are equal to twice HCLK. Programming EventFlag bit of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are performed when MemClk1 and MemClk2 are made equal to twice that of HCLK with EbiReqPos1 and EbiReqPos2 are set in the EbiTrCntl register.

- Request asserted and deasserted by Port1 and Port2 simultaneously.
- Request asserted and deasserted by Port1 and Port3 simultaneously.
- Request asserted and deasserted by Port2 and Port3 simultaneously.
- Request asserted by all the three Ports simultaneously and deasserted sequentially.
- Request asserted sequentially and deasserted simultaneously.
- Backoff mechanism on Port 1 is checked.
- Request asserted by two Ports for single clock cycle.
- Request for all three ports raised simultaneously, Port 1 and Port 2 timeout values loaded are same, when Port 2 releases the request the least recently used algorithm is checked.

7.1.6 EBIMck1Mck3Eq2HCLK Test

In this test case the MemClk1 and MemClk3 is equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMck1Mck3Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk1 and MemClk3 of EbiTrClk register present in the trick box is programmed in order to achieve MemClk1 and MemClk3 are equal to twice HCLK. Programming EventFlag bit of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are

performed when MemClk1 and MemClk3 are made equal to twice that of HCLK with EbiReqPos1 and EbiReqPos3 bit is set in the EbiTrCntl register.

- Request asserted and deasserted by Port1 and Port3 simultaneously.
- Request asserted and deasserted by Port1 and Port2 simultaneously.
- Request asserted and deasserted by Port2 and Port3 simultaneously.
- Request asserted by all the three Ports simultaneously and deasserted sequentially.
- Request asserted sequentially and deasserted simultaneously.
- Backoff mechanism on Port 1 is checked.
- Request asserted by two Ports for single clock cycle.
- Request for all three ports raised simultaneously, Port 1 and Port 3 timeout values loaded are same, when Port 2 releases the request the least recently used algorithm is checked.

7.1.7 EBIMck2Mck3Eq2HCLK Test

In this test case, MemClk2 and MemClk3 are equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIMck2Mck3Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk2 and MemClk3 bits of EbiTrClk register present in the trick box is programmed in order to achieve MemClk2 and MemClk3 are equal to twice HCLK. Programming EventFlag bit of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following conditions are performed when MemClk2 and MemClk3 are equal to twice that of HCLK with EbiReqPos2 and EbiReqPos3 bit is set in the EbiTrCntl register.

- Request asserted and deasserted by Port2 and Port3 simultaneously.
- Request asserted and deasserted by Port1 and Port2 simultaneously.
- Request asserted and deasserted by Port1 and Port3 simultaneously.
- Request asserted by all the three Ports simultaneously and deasserted sequentially.
- Request asserted sequentially and deasserted simultaneously.
- Backoff mechanism on Port 2 is checked.
- Request asserted by two Ports for single clock cycle.
- Request for all three ports raised simultaneously, Port 2 and Port 3 timeout values loaded are same, when Port 1 releases the request the least recently used algorithm is checked.

7.1.8 EBIM1M2M3Eq2HCLK Test

In this test case, MemClk1, MemClk2 and MemClk3 are equal to twice that of HCLK. Programming the EbiTrCntl register with different values raises the different combinations of request.

Test Identification No: EBIM1M2M3Eq2HCLK

Apply HRESETn. Program some values in the EbiTrAddr and EbiTrData registers corresponding to each port present in the trick box. Program some values in the counter register corresponding to each port present in the trick box. MemClk1, MemClk2 and MemClk3 bits of EbiTrClk register present in the trick box is programmed in order to achieve MemClk1, MemClk2 and MemClk3 are equal to twice HCLK. Programming EventFlag of EbiTrCntl register in the trick box high enables the event flag. Programming the EbiTrCntl register present in the trick box with different values raises different combination of request. External bus granting and backoff mechanism checks for the following

conditions are performed when MemClk1, MemClk2 and MemClk3 are made equal to twice that of HCLK with EbiReqPos1, EbiReqPos2 and EbiReqPos3 bit set in the EbiTrCntl register.

- Request asserted and deasserted by Port1.
- Request asserted and deasserted by Port2.
- Request asserted and deasserted by Port3.
- Request asserted and deasserted by all the three Ports simultaneously.
- Request asserted sequentially and deasserted simultaneously.
- Request asserted by all the three Ports for a single clock.
- Request asserted by all the three Ports simultaneously, Counter Registers of Port1 and Port2 are loaded with same timeout values to check the Backoff Generation mechanism and also the least recently algorithm.

7.1.9 EBIRandom Test

In this test case the MemClk1, MemClk2 and MemClk3 are set randomly.

Test Identification No: EBIRandom Test

Apply HRESETn.

External bus granting and backoff mechanism checks for the following conditions are performed:

- Requests are raised randomly.
- Wait cycles are inserted randomly.
- Counter registers are loaded randomly.
- EbiReqPos bits are set randomly.
- Memory clock is set randomly.

This Test takes care of some conditions and corner cases, which are missed in Directed Test cases.